

InMoov'23

Development of a 3d printed assistive robot.

Submitted By:

Ahmed Mohamed Ibrahim
Mohamed Abdelaziz Mohamed
Mohamed Adel Abd El Aaty
Mohamed Hossam Mohamed
Mohamed Magdy Salem
Nooreldin Nasser Hussein

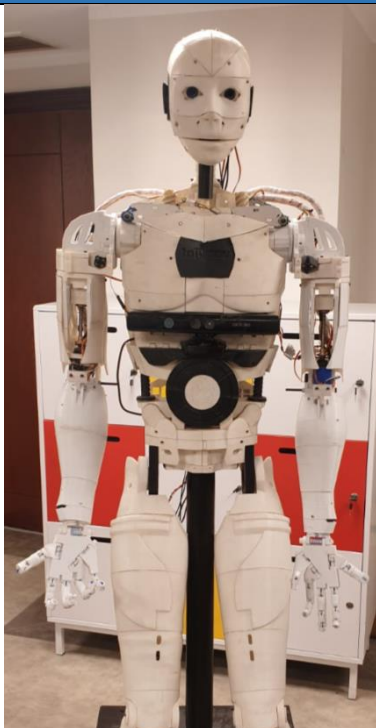


**Mechanical Engineering
Department,
Mechatronics Section**

Graduation Project Report

**Supervisor(s):
Mohamed Ibrahim Awad
Shady Ahmed Majed**

**Date
Jan 2023**





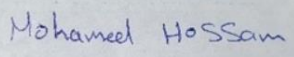
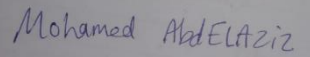
1 Elsarayat St., Abbaseya, 11517 Cairo, Egypt
Fax: (+20 2) 26850617
www.eng.asu.edu.eg



DECLARATION

We/I hereby certify that this Project submitted as part of our/my partial fulfilment of BSc in ***(Mechatronics, Robotics and Automation Engineering)*** is entirely our/my own work, that we/I have exercised reasonable care to ensure its originality, and does not to the best of our/my knowledge breach any copyrighted materials, and have not been taken from the work of others and to the extent that such work has been cited and acknowledged within the text of our/my work.

Signed: by all students

Name: Ahmed Mohamed Ibrahim	Name: Mohamed Adel Abd El Aaty
Signature: 	Signature: 
Name: Mohamed Magdy Salem	Name: Mohamed Hossam Mohamed
Signature: 	Signature: 
Name: Mohamed Abdelaziz Mohamed	Name: Nooreldin Nasser Hussein
Signature: 	Signature: 

Date: 31st of January 2023



ACKNOWLEDGMENT

We would like to express my special thanks to our mentors Dr/ Mohamed Ibrahim & Dr/ Shady Ahmed for the time and the efforts provided throughout the year. Their useful advice and suggestions were really helpful to the team during the project's completion. In this aspect, we are eternally grateful to you.

Acknowledgement to HCM Lab (Human Centered Mechatronics Lab), EMP (Extended Modular Program) and Academic Institute for Heart Surgery at Ain Shams University for allowing us to work on this project providing us with a suitable working environment.

We also would like to thank Dr/ Tamer Emara for his valuable time and all the facilities given to us to enhance our working environment.

In addition, a special thanks for Eng. Hamdy Osama for his time and technical advice that helped us to understand more the concepts of the 3d printing process that led to a better results of the printed parts

Also, the robot is a piece of collaborative work done by many teams over the past years so, we would like to thank the previous InMoov Ain Shams teams. especially Eng. Abanoub Ramsis of the 2022 team. as through their aid and help we were able to continue working on InMoov Robot

Finally, much thanks to our families for their consistent support.



ABSTRACT

The following pages describe the journey of the InMoov'23 team to enhance and developing the current form of the robot.

InMoov is an Autonomous, Open Source, 3D Printed Life-Size Robot. The robot has already been 3D printed previously by different groups of Ain Shams University students as their graduation project. Was then optimized by the 2022 by Using ROS (Robot Operating System) as its main Software platform to map the surrounding environment, localize itself and then navigate and avoid some obstacles.

Face and object detection features were also added with sound localization to reach the function of an interactive humanoid robot.

The required Task of this year's team so far was to assure a proper operation of the robot with all the mentioned existed features. To fulfil this requirement lots of the previously 3d printed parts were reprinted again, many of the motors were replaced. the software itself took a decent amount of time to be understood and for us to be able to operate much of it by ourselves regarding the tight time frame we had. and the lack of previous experiences with ROS

TABLE OF CONTENTS

Table of FIGURES.....	2
list of tables.....	4
Nomenclature	5
CHAPTER ONE: INTRODUCTION	6
CHAPTER TWO: SYSTEM DESIGN	7
2.1 Product Design Requirements	7
2.2 Design process.....	9
CHAPTER THREE: HARDWARE	10
3.1 LIST OF HARDWARE COMPONENTS.....	10
3.2 SYSTEM ACTUATORS	11
3.3.1 Sizing Calculations	12
3.3.2 Selection Criterion	15
3.3.3 Servomotor Hacking.....	16
3.4 Wiring.....	17
3.4.1 Circuit Schematic.....	17
3.4.2 wiring current state	19
3.4.3 Harness Design	19
3.4.4 Wiring Board & Wire Management	20
3.5 BATTERY SIZING.....	21
3.6 Mechanical modifications.....	22
CHAPTER FOUR: SOFTWARE	26
4.1 InMoov as Autonomous system	26
4.1.1 Control architecture:	26
4.1.2 Locomotion.....	28
4.1.3 localization:	29
4.1.4 Mapping:.....	37
4.1.5 Navigation and path planning:	39
4.2 DETECTION SYSTEMS.....	46
4.2.1 Sound detection and location	46
4.2.2 Object Recognition	48
4.2.3 Sign recognition:.....	62
4.2.3 Speech Recognition	64
4.3 Robot kinematics.....	68
CHAPTER FIVE: COST ANALYSIS	80
CHAPTER SIX: CONCLUSION	81

TABLE OF FIGURES

Figure 1: Product Design Requirements	7
Figure 2 Engineering Design Process	9
Figure 3.....	11
Figure 4.....	11
Figure 5.....	11
Figure 6.....	11
Figure 7.....	11
Figure 8 Arm Weight = 2000g.....	12
Figure 9 InMoov Arm Length = 80cm	12
Figure 10 60Kg.cm servo motor	15
Figure 11 differences in dimensions between the 60kg.cm and the 35kg.cm.....	15
Figure 12 InMoov'22 team hacking a servo motor	16
Figure 13 the circuit after removing the potentiometer.	16
Figure 14 Full servos connection schematic.	17
Figure 15 Base Schematic.	17
Figure 16 Servo I2C board #1.	18
Figure 17 Servo I2C board #2.	18
Figure 18 flat ribbon cables	19
Figure 19 arms harness	20
Figure 20 the wiring at the back of the robot.....	20
Figure 21 the main components board.....	20
Figure 22 Battery sizing	22
Figure 23 broken face.	22
Figure 24 broken arm cover.....	22
Figure 25 broken arm cover with screws.....	22
Figure 26 broken arm cover.....	23
Figure 27 broken finger.	23
Figure 28 broken arm.	23
Figure 29 broken elbow.....	23
Figure 30 broken shoulder gear.	23
Figure 31 all broken parts.....	23
Figure 32 Right arm assembly.....	24
Figure 33 Left arm assembly.....	24
Figure 34 biceps assembly.....	24
Figure 35 Head assembly.....	24
Figure 36 biceps assembly.....	25
Figure 37 Assembled full arms.....	25
Figure 38 ROS facilities.	26
Figure 39 ROS version used.....	26
Figure 40 roserial_python node	27
Figure 41 Robot Wheel setup	28
Figure 42.....	28
Figure 43 traditional dead reckoning.....	29
Figure 44 Dead reckoning in robotics.....	30
Figure 45 Rotary encoder	30
Figure 46 2-channel encoder	31
Figure 47 direction determination.....	31
Figure 48 wrong Axis of rotation	33
Figure 49 resulted Topics.	33
Figure 50 Mobile application.....	33
Figure 51 script that handles the mobile readings	34
Figure 52 IMU mobile trial.....	34
Figure 53 wrong resulted tf tree.....	35
Figure 54 IMU PCB layout.....	35

Figure 55 Robot Pose EKF	36
Figure 56 Kinect calibration	37
Figure 57 Mapping Process	38
Figure 58 rqt_steering node interface	39
Figure 59 path planning.....	39
Figure 60 Path planner inputs and outputs.....	40
Figure 61 Optimization techniques.....	41
Figure 62 ROS navigation Stack	44
Figure 63 move base configurations.....	44
Figure 64 Costmap.....	45
Figure 65 Costmap layers	45
Figure 66 microphone sensor.....	46
Figure 67 Sound localization mechanism.....	47
Figure 68 sensors positioning	47
Figure 69 sensors locations on the robot	47
Figure 70 Object detection classification.....	48
Figure 71 Confusion Matrix	50
Figure 72 Intersection over union.....	51
Figure 73 COCO for YOLOv3	52
Figure 74 YOLO timeline.....	53
Figure 75 YOLO vs other detectors.....	54
Figure 76 YOLO v3 Performance	54
Figure 77 CNN layers.....	55
Figure 78 CNN extraction and classification.....	56
Figure 79 Detection steps	58
Figure 80 Detection in action	59
Figure 81:Darknet-53	60
Figure 82 Detection in ROS	61
Figure 83 Sign recognition	62
Figure 84:Sign detection testing	62
Figure 85:Sign detection testing	63
Figure 86.....	64
Figure 87 Speech to Text.....	64
Figure 88.....	65
Figure 89 Google Bard	65
Figure 90.....	66
Figure 91.....	69
Figure 92.....	70
Figure 93.....	71
Figure 94.....	72
Figure 95.....	73
Figure 96.....	73
Figure 97.....	74
Figure 98.....	75
Figure 99.....	75
Figure 100.....	76
Figure 101.....	76
Figure 102.....	77
Figure 103.....	77
Figure 104.....	78
Figure 105.....	79



LIST OF TABLES

Table 1 Hardware Components	10
Table 2 Components rated current and voltage.	21
Table 3 Battery capacity and power required.	21
Table 4 Comparison between path planning technique	42
Table 5 Speech Commands	67
Table 6 Cost Analysis.....	80

NOMENCLATURE

ACRONYM	Definition of Acronym
ROS	Robot Operating System
I2C	Inter-Integrated Circuit
3D	Three Dimensions
AI	Artificial Intelligence
DC	Direct current
PWM	Pulse Width Modulation
HD	High Definition
IMU	Inertial measurement unit
PCB	Printed Circuit Board
FR	Front Raise
LR	Lateral Raise
URDF	Unified Robotics Description Format
SLAM	Simultaneous localization and mapping
msg	Message published by ROS publisher to a ROS topic



CHAPTER ONE: INTRODUCTION

InMoov is the first Open-Source 3D printed life-size robot. It began as a personal project of Gael Langevin, a sculptor and designer. InMoov is thought as a development platform for universities, laboratories, and makers. The robot is replicable on any home 3D printer, and the development of the software is based on sharing by forums, the InMoov website and the comments there. That means that with the same base, there are countless projects throughout the world, since the final purpose of the robot is up to the person who is creating it.

This project is very useful for the undergraduate students because there are many engineering aspects involved into this project to be developed such as: mechanical design, electronics, circuits, wiring, AI, hardware, and software engineering. It also has a utility for the university, as the final purpose of the robot can serve as a presentation forum of the university. Finally, working on this project is helpful for the InMoov community, as the problems and the solutions found are shared in the forums.

The final goal of the project is to develop a fully autonomous assistive robot dedicated to the medical field. Reducing the amount of direct interaction between the medical staff and the highly infectious cases. Thus, reducing the risk on the staff's health and reducing the probability of transmitting diseases overall.

CHAPTER TWO: SYSTEM DESIGN

The vision of this years' InMoov team is to develop the already existing form of the robot into a fully autonomous robot dedicated to the medical field. Which will be later sold as a product for hospitals and health care foundations as a next step. This product has to meet the product design pillars to be able to compete in the market.

2.1 PRODUCT DESIGN REQUIREMENTS

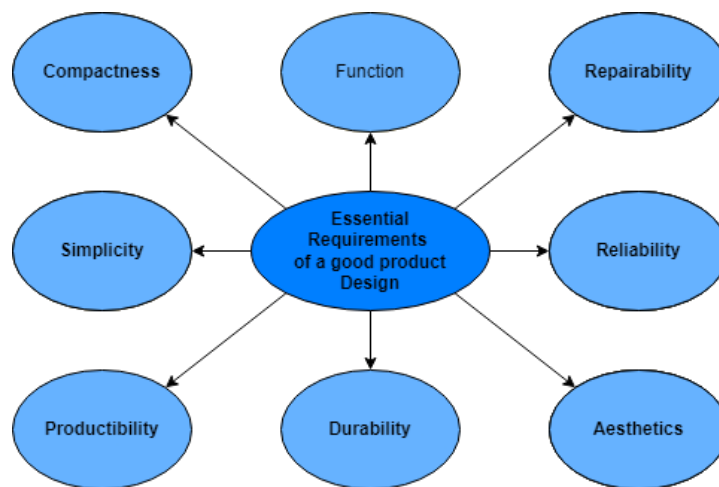


Figure 1: Product Design Requirements

Functionality:

Functionality refers to whether a design works and helps the users meet their goals and needs. From products such as chairs or tables to designs such as books or web interfaces, functionality varies but is everywhere. When a design is highly functional, it does what it's expected to do and does it well. Good design has a purpose and is crafted in such a way that it will consistently and reliably execute a particular function.[1]

Simplicity:

Simplicity in a product is about helping the user solve a problem or achieve a goal in an easier, faster, and better way. The product must have the least number of operations without affecting its functionality.[2]

Reliability:

Reliability of a product conveys the concept of dependability, successful operation or performance and the absence of failures. It is an external property of great interest to both manufacturer and consumer. Unreliability (or lack of reliability) conveys the opposite. A more technical definition is the following:

Aesthetics:

Aesthetics is a core design principle that defines a design's pleasing qualities. It refers to how the product looks, feels, sounds, and in some cases tastes or smells. Quality must be assured in all the mentioned things to make the product able to compete at the market. For example, the packaging design should be graphically appealing and colorful, emerged with the current design trends for the product to be more appeal to the customers

Repairability:

Repairability is a measure of the degree of how easy it is for a product to be repaired, either by the consumer or the provider. The better the repairability of the produce the more value and demand by the consumers.

Compactness:

Compactness in a product is to occupy less space and having lower weight. That is one of the most trending design aspects now a days as we see many products are getting smaller such as smart phones and laptops although many products are getting larger such as the televisions and screens are getting bigger. Off course the limitations on the compact size are the functionality of the product and the ease of usage.

Durability:

Durability is the ability of the product to remain functional without the need of maintenance increasing its lifetime. Better durability attracts more consumers, when consumers buy products that costs a lot, they expect it to have a long lifetime and to not repair it constantly, which is why durability of a product is an essential aspect.

Producibility:

The product must be designed so that it can be mass produced and to be produced easily with a low cost and in a quick manner. [3]

2.2 DESIGN PROCESS

Putting in mind the mentioned design pillars. Going through a well-planned design process is the next step in our product design [4]

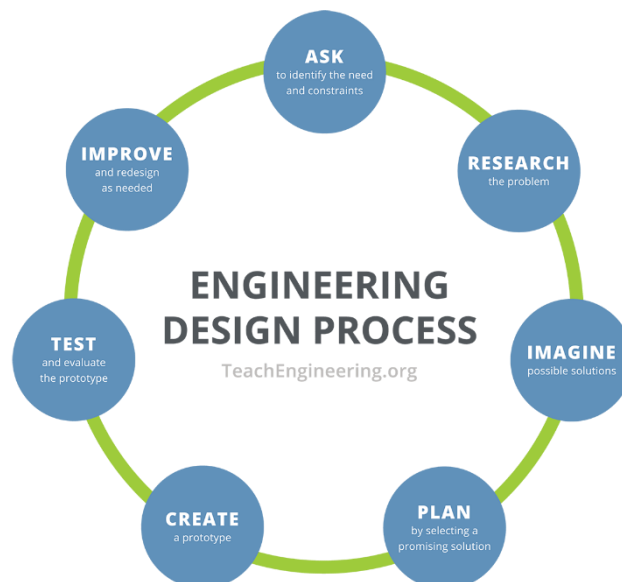


Figure 2 Engineering Design Process

CHAPTER THREE : HARDWARE

3.1 LIST OF HARDWARE COMPONENTS

no.	Component	Quantity	Function
1	3D printed parts	-	Main Body of the InMoov Robot
2	ReSpeaker 2-mics Pi HAT	1	Sound detector
3	Arduino Mega	2	Controls whole system
4	Microsoft LifeCam HD-3000	2	Responsible for objects, faces and items detection
5	Buck Regulator	2	Regulates voltages
6	I2C Servomotor Driver	2	Drives all servomotors with PWM
7	35 kg.cm Servomotor	2	Actuates Front Raise movement of the robot shoulder
8	25 kg.cm Servomotor	6	Actuates Torsional, Lateral Raise and Biceps of the robot arm
9	10 kg.cm Servomotor	12	Actuates Fingers, Wrist movements of the robot hand
10	1.3 kg.cm Servomotor	1	Actuates Eyes to a specific direction
11	Lithium-ion Battery Package	2	Power Supply to the robot upper body and Kinect Camera
12	High Grade Fishing Line	1	Acts as a belt in Pulley-Belt System that exists in InMoov Fingers
13	Dual Channel DC Motor Driver	1	Drives 2 wiper motors
14	Windshield Wiper Motor	2	Actuates robot base
15	Rotary Encoder	2	Motor Feedback
16	IMU Sensor	1	Acts as a Gyroscope and Accelerometer to the whole system
17	Xbox Kinect v1	2	One Acts as a laser sensor that feeds back laser signals to the Navigation Stack while the other used to retrieve the depth of the detected object
18	Lead-acid Battery 12V – 12 Amp	1	Power Supply to the robot base

Table 1 Hardware Components

3.2 SYSTEM ACTUATORS

The actuators used at the upper body are four varied sizes of servomotors of sizes 35 kg.cm, 25 kg.cm, 10kg.cm and a small servo of 1.3 kg.cm “figures 3, 4, 5, and 6”.

A servomotor is a device comprising of a two wire DC motor, set of gears, a position-sensing device (usually a potentiometer), an integrated circuit and output shaft. Unlike the DC motor, Servo has three wires. One is for power; one is for ground and the third wire is a control input line. Through the input line (PWM) we can send a signal that represents a desired output position of the servo shaft. The shaft does not rotate freely like in a common DC motor but can only turn maximum of specific degrees back and forth



Figure 3



Figure 4



Figure 6



Figure 5

For the Base 2 Windshield Wiper DC motors were used for actuation



Figure 7

The following sections (from 3.3.1 to 3.3.3) describes all the calculations and selections made by the InMoov'22 team

3.3.1 Sizing Calculations

Arms

Our robot has two arms with two shoulders that each have four different types of motions, moreover, a motor with a dedicated mechanism is used for the movement of every angle. These motions are:

- Front Raise of Shoulder (FR)
- Lateral Raise of Shoulder (LR)
- Torsional Motion of Shoulder
- Biceps Motion

Givens:

InMoov Arm Weight (m) = 2kg

InMoov Arm Length (l) = 80 cm

Gear System Efficiency (η) = 0.5

@ Worst Case Scenario.

In our case it was needed to size three motions: FR, LR and Biceps. Most importantly the sizing was about finding the best matching motor for the FR motion as it was crucial for our robot missions.



Figure 8 Arm Weight = 2000g



Figure 9 InMoov Arm Length = 80cm

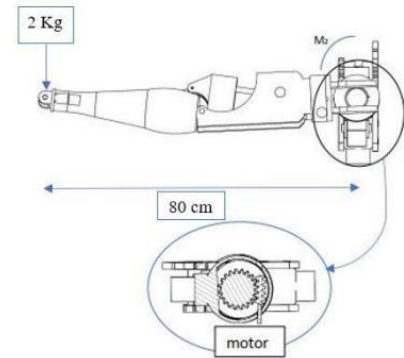
- **Front Raise:**

Gear Reduction Ratio: $(Z_A / Z_B) = 1:15$

Moment @ Central Point of Gear System (M_2) = $m * l = 2 * 80$

= 160 Kg.cm

And this value equals Torque B Then, Torque required to lift the arm



Hence, Motor Torque needed to load carry the FR motion should be ≥ 22 Kg.cm

- **Lateral Raise:**

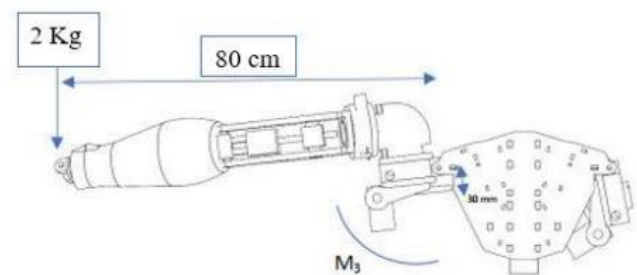
It was found that this motor's mechanism is jointed directly to a threaded shaft that is paired with a threaded nut. that forms a reduction ratio.

Gear Reduction Ratio: $(Z_A / Z_B) = 1:12$ Moment @

Central Point of R.R. System (M_2) = $m * l = 2 * 80 = 160$

Kg.cm

And this value equals Torque B. Then, Torque required to lift the arm



$$(T_A) = \frac{1}{\eta} * \left(\frac{1}{Z_A / Z_B} \right) * T_B = \frac{1}{0.5} * \frac{1}{12} * 160 = 26.67 \text{ Kg.cm}$$

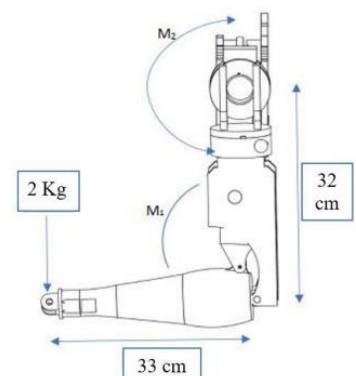
Hence, Motor Torque needed to load carry the LR motion should be ≥ 30 Kg.cm

- **Biceps:**

As LR, it was clear that the Biceps motor's mechanism is also jointed directly to a threaded shaft that is paired with a threaded nut that forms a reduction ratio.

Reduction Ratio: $(Z_A / Z_B) = 1:12$

Moment @ Central Point of R.R. System (M_2) = $m * l = 2 * 33 = 66$ Kg.cm And this value equals Torque B. Then, Torque required to lift the arm.



$$(T_A) = \frac{1}{\eta} * \left(\frac{1}{Z_A / Z_B} \right) * T_B = \frac{1}{0.5} * \frac{1}{12} * 66 = 11 \text{ Kg.cm}$$

Hence, Motor Torque needed to load carry the Biceps motion should be ≥ 15 Kg.cm

- **Head**

calculations of sizing for the servomotors of the head Motor sizing calculations for the head:

Mass = 1 Kg, assuming it is a sphere with $d=21$ cm

$$I = I_c + md^2$$

$$I = \frac{2}{5} mr^2 + mr^2$$

$$I = \frac{2}{5} * 1000 * 21^2 + 1000 * 21^2$$

$$I = 617400 \text{ gm.cm}^2 = 0.06174 \text{ Kg.m}^2$$

Assume for safety:

$$J_{motor} = J_{Load}$$

$$F = W = mg = 10N$$

$$J_{Total} = 0.12348 \text{ Kg.m}^2$$

$$T_L = mgR = 10 * 0.21 = 2.1 \text{ N.m}$$

$$T_m = T_L + J\alpha$$

Velocity Profile:

$$\theta_{Final} = 65^\circ = 1.135 \text{ rad/s}$$

$$0.5 V_{max} * 0.65 + 0.65 * V_{max} + 0.5 V_{max} * 0.65 = 1.135 \text{ rad/s}$$

$$V_{max} = 0.873$$

Acceleration Profile:

$$\alpha_{1^{st} part} = \frac{0.873}{0.65} = 1.343 \text{ rad/sec}^2$$

$$\alpha_{2^{nd} part} = 0$$

$$\alpha_{3^{rd} part} = -1.343 \text{ rad/sec}^2$$

$$\alpha_{4^{th} part} = 0$$

Torque:

$$T_{m1} = 2.1 + 0.13248 * 1.343 = 2.266 \text{ N.m}$$

$$T_{m2} = 2.1 + 0.13248 * 0 = 2.1 \text{ N.m}$$

$$T_{m3} = 2.1 + 0.13248 * 1.343 = 1.9342 \text{ N.m}$$

$$T_{m4} = 2.1 + 0.13248 * 0 = 2.1 \text{ N.m}$$

Torque Profile:

$$T_{rms} = \sqrt{\frac{2.267^2 + 2.1^2 * 0.65 + 1.93^2 * 0.65 + 2.1^2 * 0.05}{0.65 + 0.65 + 0.65 + 0.5}} = 2.265 \text{ N.m}$$

$$T_{max} = 2.265 \text{ N.m}$$

$$V = 1.135 \text{ rad/s}$$

It was found that a servo motor with 25 Kg/cm will be suitable for this application.

3.3.2 Selection Criterion

As the calculations clarified, a suitable actuator needed to run the critical motions mentioned before. Hence, a market search was done. At that time of the project, it was found that no servomotor with the required torque of FR motion was not available in Egypt whilst Amazon.US contained a group of servomotors that fits our system perfectly.

In Egypt: 25Kg.cm servomotors were found but a bigger size of torque was needed. So, the past team added a 35 Kg.cm servomotor as it has a higher load capacity and the same dimensions as 25 Kg.cm one. Depending on the previous thoughts their choice relayed on: ZOSKAY 35kg high Torque Coreless Servomotor

Why they avoided the 60 Kg.cm servomotor?

For the following reasons

1. the past team faced some problem with the servomotor hacking process as some of the motor's inner PCB were damaged during the soldering process, so they had no option of taking the risk of damaging a motor that has no alternative in the Egyptian market
2. The difference in dimensions as the 60 Kg.cm servo is much different than the 25 or the 35 ones as it comes with wider bracket cover so that it will affect the design.



Figure 10 60Kg.cm servo motor

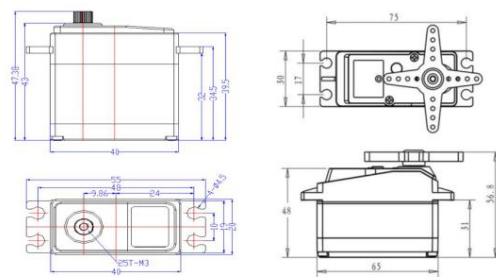


Figure 11 differences in dimensions between the 60kg.cm and the 35kg.cm

3.3.3 Servomotor Hacking

This section describes how the InMoov'22 team hacked servo motors to reach a precise position control. We recommend taking a look at InMoov'22 thesis file attached to the appendix to get more knowledge about the motor hacking process.



Figure 12 InMoov'22 team hacking a servo motor

Detailed Steps:

1. Remove the attachment from the Servo's main shaft and unscrew the four screws.
2. Remove the top cap and you will see a set of gears and an axle that will be detached.
3. Now remove the bottom cap and you can see the three wires following a circuit. We see three red wires that are connected to a potentiometer inside which we need to de-solder. We need to cut these wires from both ends (Potentiometer + Circuit). Leaving only the two DC motor wires which are connected to the circuit.
4. We need to solder the two resistors in serial connection. That gives us three pin resistors. We need to connect each pin to each knob where the three red wires were connected. It should look like this after soldering. After this, we need to fit the circuit back into place and close the bottom cap.
5. Now it is time to chop off the mechanical stopper on one of the gears using any blade.



Figure 13 the circuit after removing the potentiometer.

InMoov'21 faced a hardware problem of potentiometers that some of it was not working well with their feedback to the servomotors system. Whereas the past team (InMoov'22) managed to get a better performance by choosing a 5K Ohm pot that replaced the inner potentiometer of a stock servo.

3.4 WIRING

3.4.1 Circuit Schematic

There was no circuit schematic to follow in wiring the system which was hard diagnosing the problems and replacing components and preventing wrong connections which protects power circuits and protect the controllers, below are the schematics designed with Autodesk Eagle.

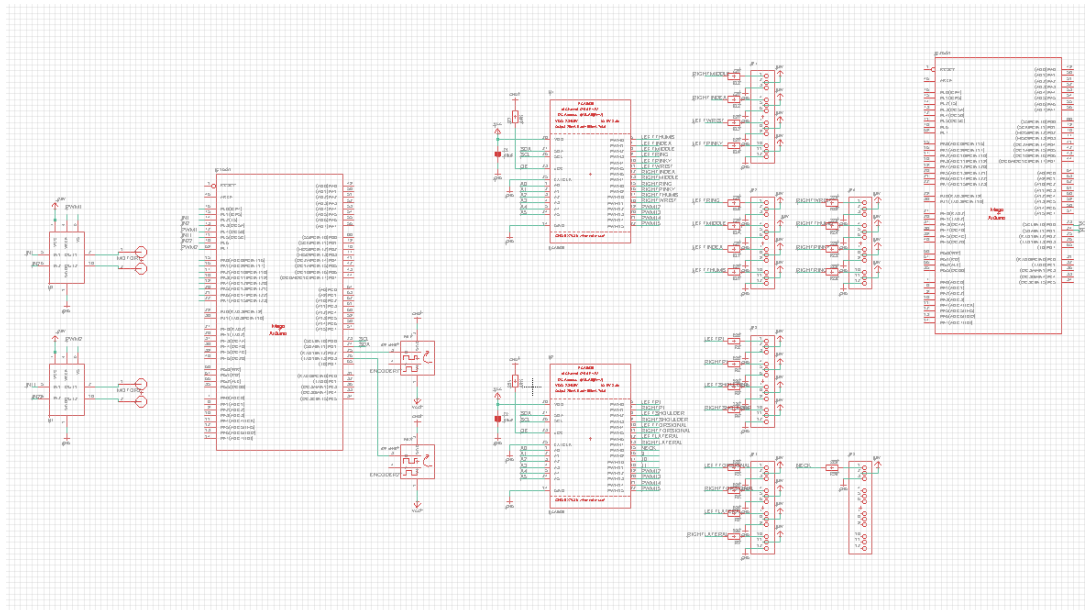


Figure 14 Full servos connection schematic.

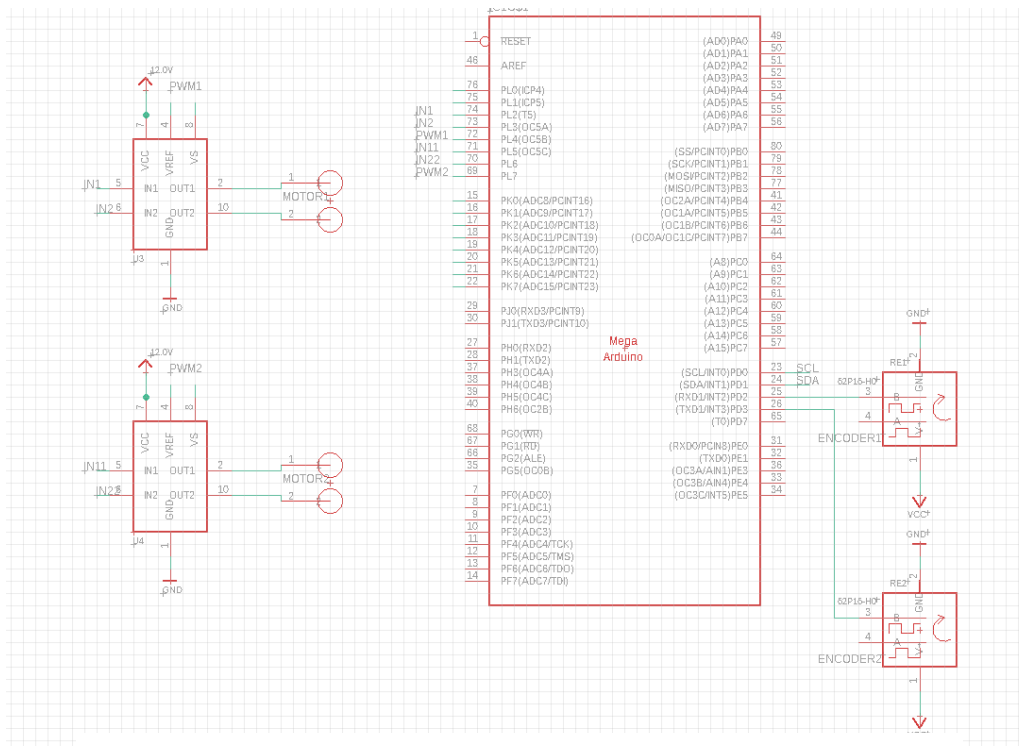


Figure 15 Base Schematic.

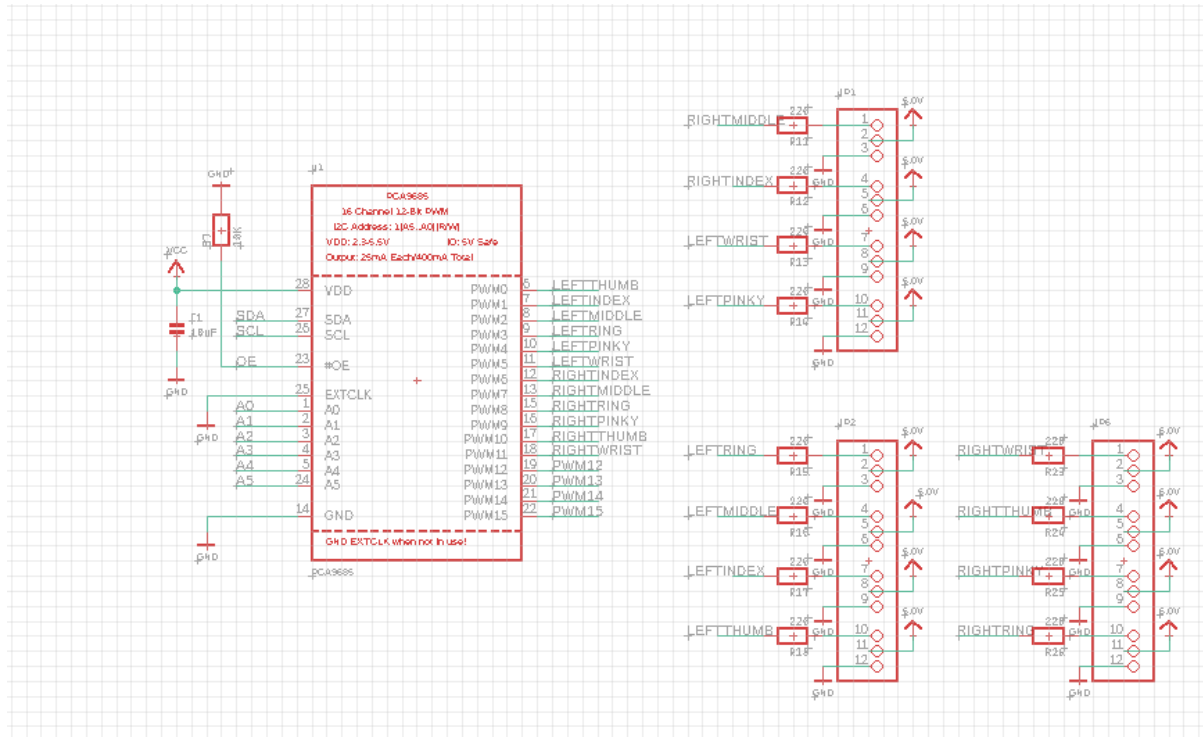


Figure 16 Servo I2C board #1.

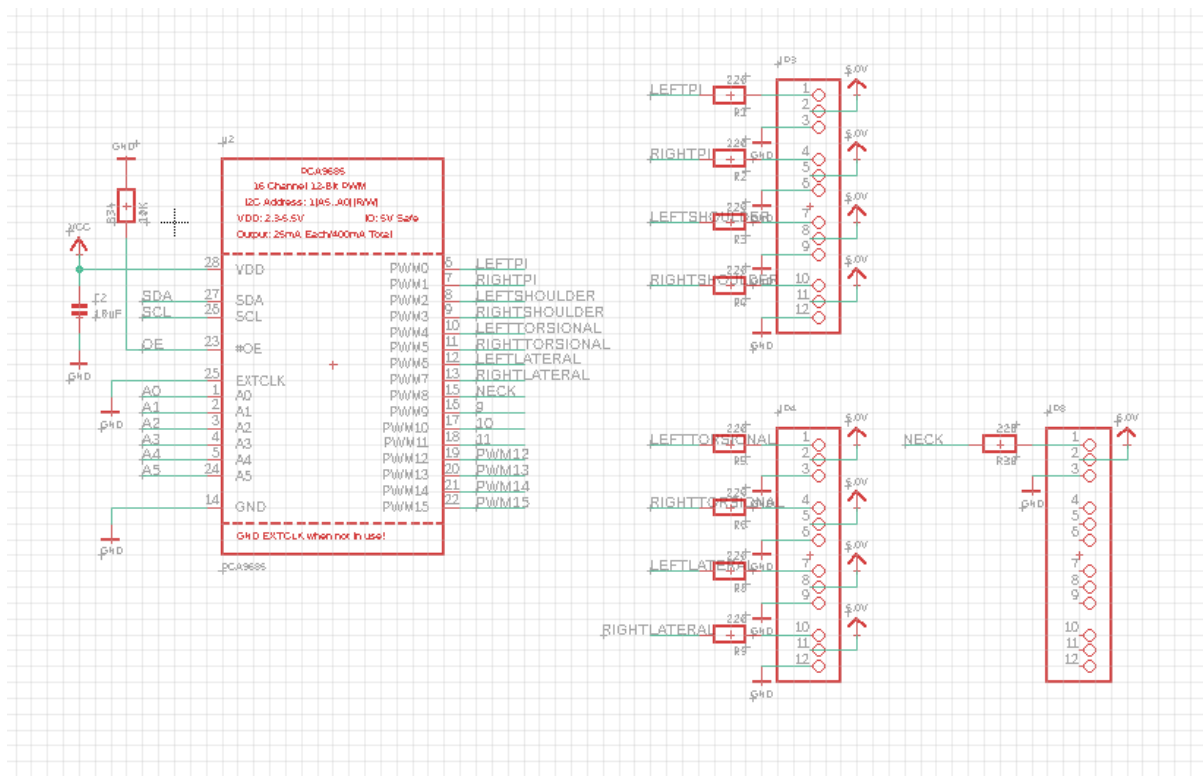


Figure 17 Servo I2C board #2.

3.4.2 wiring current state

The system contains two types of wires for two purposes: transmission of power and transmission of data:

- Power transmission lines need to be sized according to the amount of power they carry. If too small they can overheat, melt, catch fire, and generally produce all sorts of mess.
- Signal lines, in our case it is generally a PWM signal, carry little power, and size is generally not a critical issue. However nearby lines and components can produce interference through electromagnetic coupling, so attention needs to be paid to routing and shielding.
- On the other hand, we were told that noises surrounded most of the servos of the two hands, so separation was a reasonable suggestion as a solution for this issue.

3.4.3 Harness Design

The wiring harness is the main body of the network of any industrial circuit, and without the harness, there would be no reliability in these circuits. the form of the mobile robots wiring harness is basically the same, which is composed of wires, connectors, and wrapping tape. This section is specifically written to explain the wiring harnesses of our robot cables. Inspired by the cable harness design of motorsports race cars especially Formula Student race cars. [5] [6]

The following is the current harness state of the robot

Arms: It was found by the InMoov'22 team that a harness with a length = 80 cm were enough to assure that the robot arms can move in every possible direction without any tension to be happening and affect the wires.

Head: It was assumed that 50 cm of harness length were enough.

2 Types of cables were mounted on the robot.

- Flat Ribbon Cables – 36 Pins
- 2.5mm Electrical Wire (with colour code: Black, Yellow, and Red).



Figure 18 flat ribbon cables



Figure 19 arms harness

3.4.4 Wiring Board & Wire Management

The main components were tightened to the back of InMoov robot. Using a Strain relieving technique to reduce the amount of stress at a wire connection. Proper strain relief will prevent the connector from becoming unplugged, or from having the wires break loose from the connector itself. board is very configurable and maintainable as it collects both of I2C Servo drivers with a buck regulator and an Arduino to control the motors

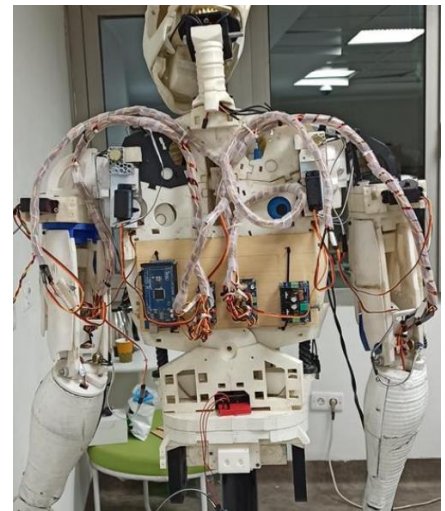


Figure 20 the wiring at the back of the robot

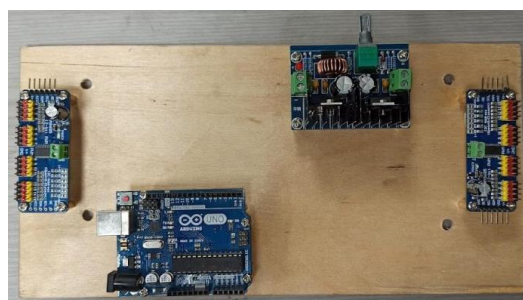


Figure 21 the main components board

3.5 BATTERY SIZING

we can specify its component into power and voltage category:

- 5V ~ 6.8 V
- 9 V
- 12 V

It means that a 12V battery would be suitable with a buck convertor to meet the required voltages.

Note: the following calculations were made by InMoov'22 team taking a Raspberry Pi 4 into account as the plan was to use it as the main processing element in the control process.

However, this plan was scrapped later.

Component	Quantity	Volt (v)	Current (A) Max
Servo 35 kg.cm	2	6.8	3
Servo 25 kg.cm	8	6.8	2.3
Servo 11 kg.cm	16	6.8	1.2
Dc Motor	2	12	2.3
Kinect	1	12	1.1
Arduino Mega	2	12	0.5
Raspberry pi 4	1	5	3
Encoder	2	5	0.1
IMU	1	5	0.1
Bluetooth	1	5	0.1

Table 2 Components rated current and voltage.

Component	Current (A) Operating	Time (min)	Time (hour)	Capacity (Ah)	Power (Watt)
Servo 35 kg.cm	2	1.5	0.025	0.1	13.6
Servo 25 kg.cm	1.5	1.5	0.025	0.3	10.2
Servo 11 kg.cm	1	1.5	0.025	0.4	6.8
Dc Motor	2.3	4.5	0.075	0.345	27.6
Kinect	1.1	15	0.25	0.275	13.2
Arduino Mega	0.5	15	0.25	0.25	6
Raspberry pi 4	3	15	0.25	0.75	15
Encoder	0.1	15	0.25	0.05	0.5
IMU	0.1	15	0.25	0.025	0.5
Bluetooth	0.1	15	0.25	0.025	0.5

Table 3 Battery capacity and power required.

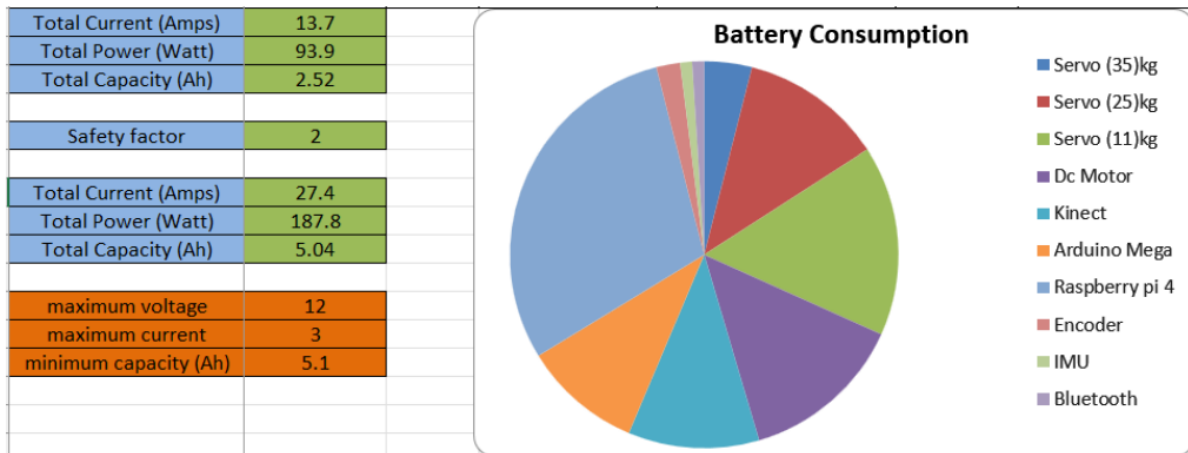


Figure 22 Battery sizing

Minimum requirements could be afforded by a battery with a capacity of 2600mAh that can supply 28 Amps. The selection was a Lithium-Ion Battery.

3.6 MECHANICAL MODIFICATIONS

The InMoov 22' Team had some mechanical problems where the parts were not meant to be disassembled and were easily broken due to bad fitting and assembly and any wrong movement will lead to a broken part. Now we reprinted the parts many times due to some inaccuracies in printing and assembled it to get a rigid and intact system.

- Here are some examples of the parts that broke either from wrong movement or assembly.



Figure 25 broken arm cover with screws.



Figure 24 broken arm cover.



Figure 23 broken face.



Figure 27 broken finger.



**Figure 28
broken arm.**



Figure 26 broken arm cover.



**Figure 30 broken
shoulder gear.**



Figure 29 broken elbow.



Figure 31 all broken parts.

- Here are the new assembled parts.

Mechanically ready parts with an assembly that can be easily disassembled without being broken and work flawlessly.

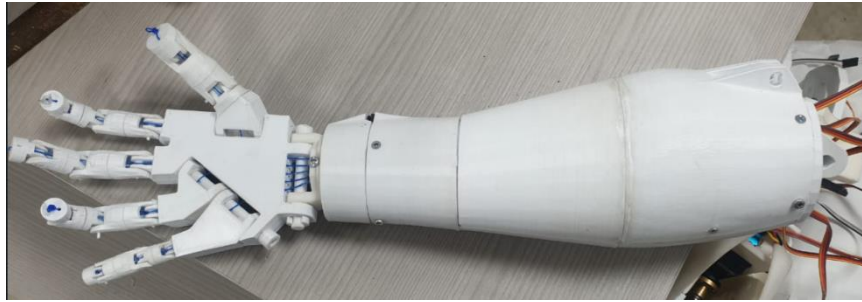


Figure 32 Right arm assembly.

Using a High-Grade Fishing Line in the pulley system in the fingers allowing them to move by rotating the servo motors.



Figure 33 Left arm assembly.

Assembled covers that are not glued to the whole body which helps in disassembly process and gives an elegant shape and allows further modifications.



Figure 34 biceps assembly.

removing the screws which nevertheless gave the assembly a better shape also gave it better assembly and disassembly process for future upgrades.



Figure 35 Head assembly.

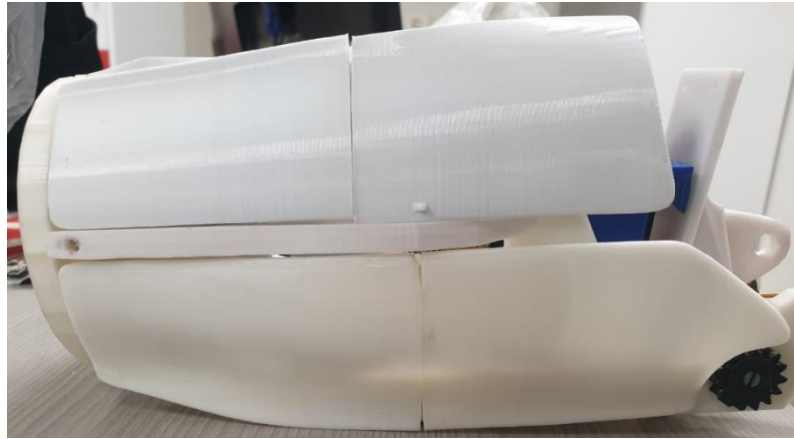


Figure 36 biceps assembly.

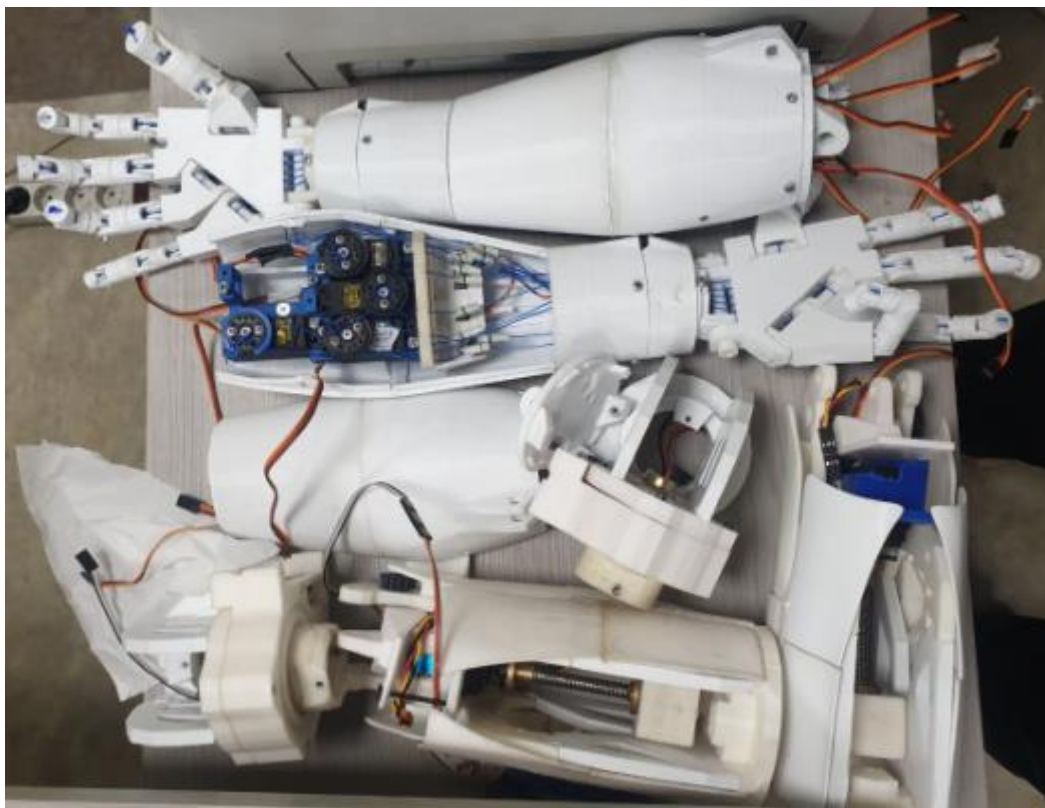


Figure 37 Assembled full arms.

Now we have a better shaped and assembled parts that will help in doing the intended functions of the humanoid robot and allow future upgrades and maintenance without breaking any part as the screws are replaced with bolts and nuts with the specific sizes for each part, the covers that needs glue are glued in a way that will not interfere with the disassembly process.

This process of reprinting and assembly took a handful of time which is worth the time, the printing took many trials to get the parameters for the printer and get better parts, while assembling; the parts needed more work on them as sanding and this is a result of the bad printing quality which resulted in more money and time waste.

CHAPTER FOUR: SOFTWARE

4.1 INMOOV AS AUTONOMOUS SYSTEM

challenges facing autonomous system

In this section we tackle the 5 main challenges facing any autonomous system development process:

- Control architecture
- Locomotion
- Localization
- Mapping
- Navigation

4.1.1 Control architecture:

Our control architecture for the autonomous robot relied on ROS (Robot Operating System) (noetic version) to achieve its functionality. ROS is an open-source software framework that provides a collection of libraries and tools for building complex robotics systems. ROS allowed us to efficiently integrate different software components, such as perception, control, and mapping, to create a unified system for controlling our autonomous robot.

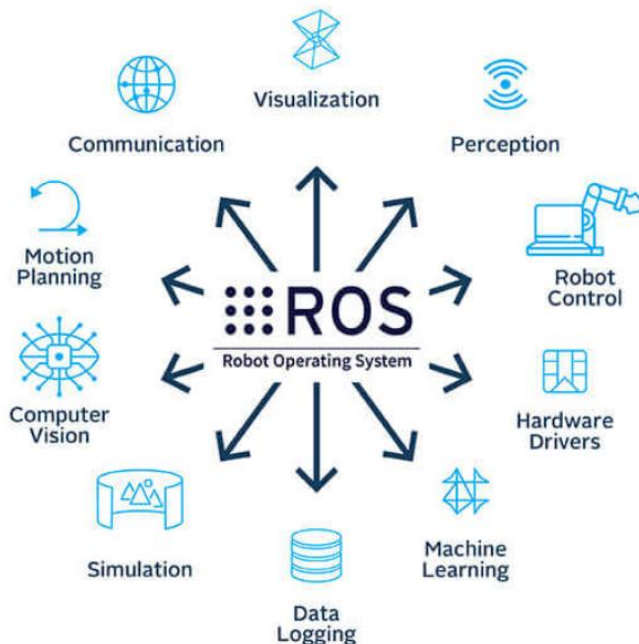


Figure 38 ROS facilities.

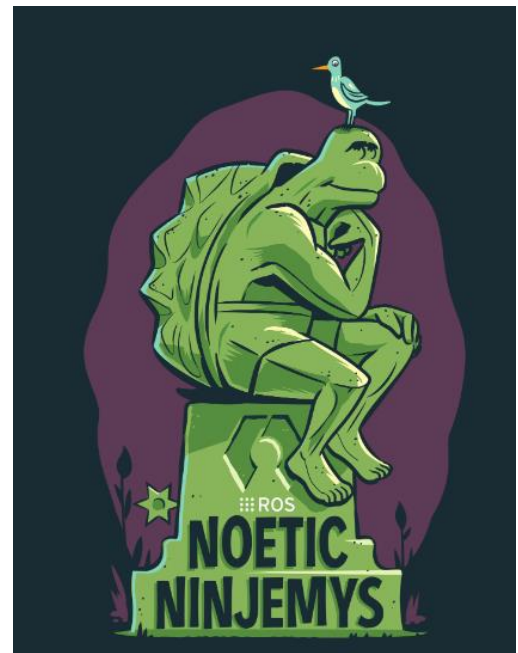


Figure 39 ROS version used.

One of the key benefits of using ROS was its modular architecture, which allowed us to develop and test each component of the control system independently. This enabled us to quickly iterate on different parts of the system and make changes as needed. ROS also provided a range of useful tools for debugging and visualization, making it easier to understand and analyze the behavior of our autonomous robot.

In addition, ROS provided a range of pre-built libraries and algorithms for common robotics tasks, such as navigation and mapping. This allowed us to leverage existing code and focus our efforts on developing custom algorithms and integrations specific to our robot and application.

The roserial library of ROS (Robot Operating System) was a key component in establishing communication between our sensors and drivers, which were connected to Arduinos, and the laptop that had ROS installed on it.

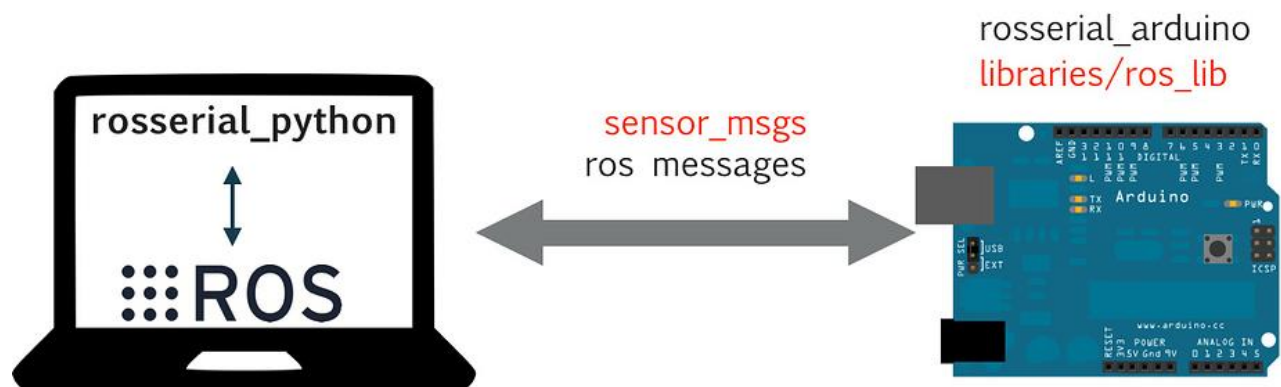


Figure 40 roserial_python node

In our project, we used Arduinos to interface with our sensors, which included encoders, IMU, sound detectors and potentiometers. As well as the motor drivers The Arduinos were connected to the laptop running ROS, and the roserial library was used to establish communication between the microcontrollers and the ROS system.

4.1.2 Locomotion

Locomotion refers to the ability of an autonomous system to move through its environment. This can include movement on wheels, legs, or other types of locomotion systems. Locomotion is typically controlled by a system of motors and actuators that allow the robot to move in response to commands from its control system.

Our autonomous robot uses a drivetrain setup with two driving wheels and two motors at the back, and a caster wheel at the middle of its front side. This setup is commonly known as a two-wheel differential drive, and it is a popular configuration for small autonomous robots.

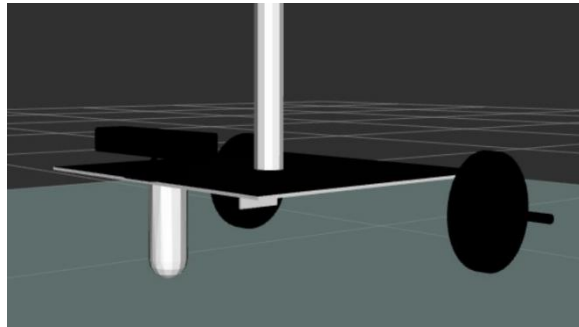


Figure 41 Robot Wheel setup

The advantages of using a two-wheel differential drive include:

1. **Maneuverability:** The two-wheel differential drive configuration provides excellent maneuverability, allowing the robot to rotate in place and navigate through tight spaces.
2. **Simple control:** The control system for a two-wheel differential drive is relatively simple, requiring only two motors to control the robot's movement and orientation.
3. **Stability:** The caster wheel provides stability to the robot, ensuring that it remains upright and balanced while in motion.
4. **Efficiency:** The two-wheel differential drive is an efficient configuration, requiring less power than other types of drivetrains, which can be useful for applications where power consumption is a concern.

In addition, many online ROS tutorials and examples work on similar two-wheel differential drive setups, making it a popular choice for robotics enthusiasts and beginners. This means



Figure 42

that there is a wealth of resources available for developers looking to build and program robots with this type of drivetrain.

4.1.3 localization:

Robot localization is the process of determining the position and orientation of a robot in its environment. simply answering the question of “where am I?”

It is a fundamental problem in robotics and is necessary for robots to navigate and perform tasks autonomously. Localization can be done using various sensors such as cameras, Lidars, and sonars.

While working on this project, we used 2 different localization techniques to develop 2 different features of the robot. Both of them depend on the dead reckoning technique. - more about this technique in the upcoming section-.

THE DEAD RECKONING TECHNIQUE:

Also known as the odometry technique, Dead reckoning is a localization technique that estimates the current position of an object by using its previous position, speed, and direction of motion. It assumes that the object moves in a known manner and uses this information to update its position estimate. Dead reckoning is commonly used in navigation systems, such as ships and aircraft, where it can provide an estimate of position when other methods are unavailable, such as in poor visibility conditions.

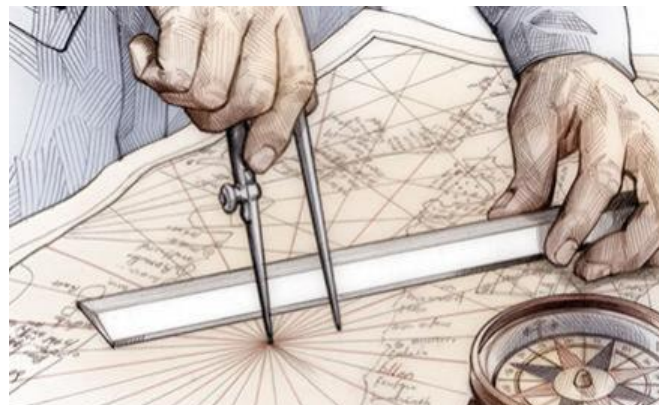


Figure 43 traditional dead reckoning

In robotics, dead reckoning is used as a localization technique to estimate the robot's position based on its previous position and motion information. This method is often used in combination with other sensors such as encoders, gyroscopes, and accelerometers to provide a more accurate estimate of the robot's position. The encoders measure the rotation of the robot's wheels, while the gyroscopes and accelerometers measure the robot's angular and linear velocity, respectively.

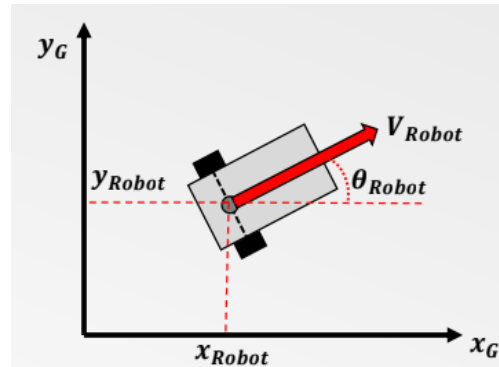


Figure 44 Dead reckoning in robotics

THE SENSORS USED TO ACHIEVE THIS TECHNIQUE:

1- Rotary Encoders:

Rotary Encoders are sensors that detect position and speed by converting rotational mechanical displacements into electrical signals and processing those signals.

The used encoder is an incremental one with 600 pulses per revolution, with 2 channels. To determine both the direction and the distance moved



Figure 45 Rotary encoder

The theory of operation comes as the following:

the shaft is connected to a disk with evenly spaced slots or windows around its circumference. A light source and a photodetector are positioned on opposite sides of the disk. As the shaft rotates, the slots or windows pass between the light source and the photodetector, causing the signal to alternate between high and low.

The number of pulses generated per revolution depends on the number of slots or windows on the disk. (In our case 600 slots on the disk = 600 pulses per full cycle)

For the determination of the direction the disk has 2 different sets (channels) of slots

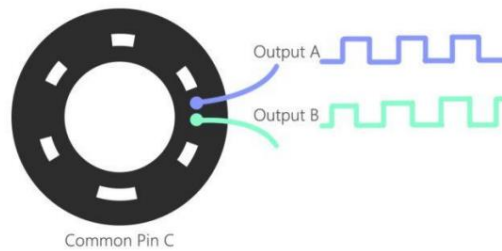


Figure 46 2-channel encoder

The two channels are typically labeled A and B, and they generate square wave signals that are out of phase (with 90 degrees at our case) with each other. As the shaft rotates, the signals from channels A and B change in a specific pattern that allows the direction of rotation to be determined.

When the shaft rotates in one direction, the signal from channel A leads the signal from channel B. When the shaft rotates in the other direction, the signal from channel B leads the signal from channel A. This phase relationship between the two channels is known as the quadrature signal.

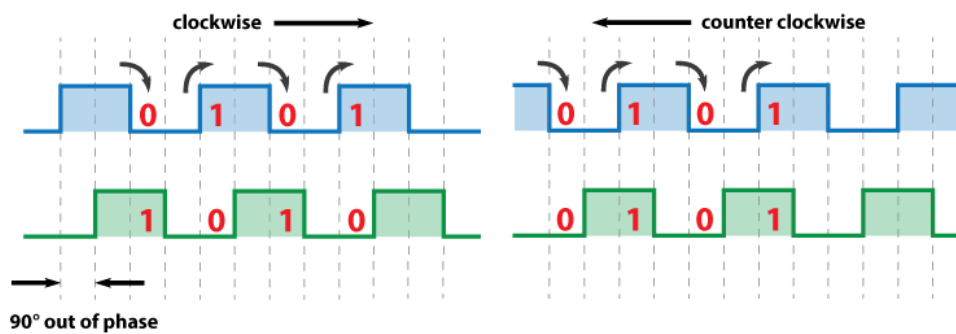


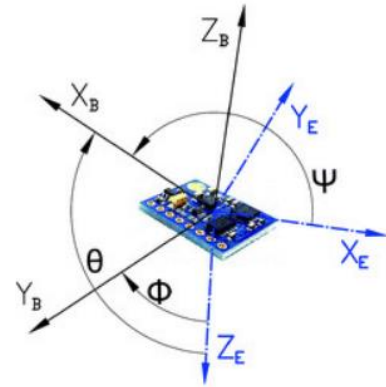
Figure 47 direction determination

2- The IMU sensor

inertia measurement unit also known as IMU.

electronic device used to calculate and reports an exact force of body, angular rate as well as the direction of the body, which can be achieved by using a blend of three sensors like Gyroscope, Magnetometer, and Accelerometer.

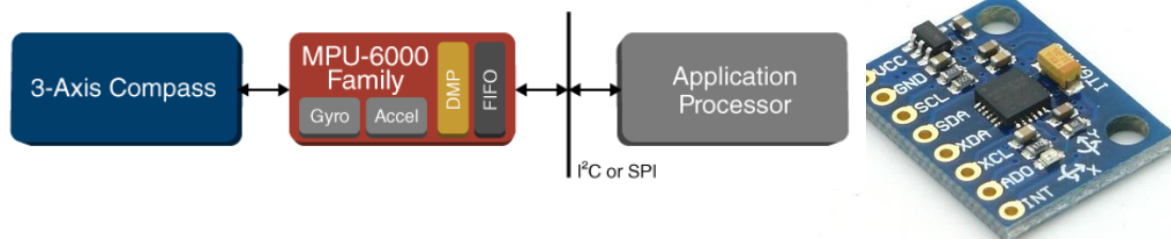
- Accelerometer: measures velocity and acceleration.
- Gyroscope: measures rotation and rotational rate.
- Magnetometer: establishes cardinal direction (directional heading).
- It also has a temperature sensor but it wasn't used in our system



An IMU sensor unit working can be done by noticing linear acceleration with the help of one or additional accelerometers & rotational rate can be detected by using one or additional gyroscopes. Some also contain a magnetometer which can be used as a heading reference. This sensor includes some usual configurations which include gyro, one accelerometer, and magnetometer for each axis used for each of the 3-vehicle axes like roll, yaw, and pitch.

IMU issues

the module used for our project was the MPU6050 manufactured by TDK. Though it wasn't cheap (costs 250 EGP), we faced multiple issues to get the readings out of it at many previous projects and the case wasn't different at this one.



- The first issue is that the tutorial we followed used Jetson Nano controller and connected the sensor directly to the I2C pins of the controller. However, that wasn't the case at our project as we used an external processing unit (Arduino UNO). We needed a sort of a bridge between the external processor and the main processor (the laptop operating ROS)

This issue was solved by using a ROS package thankfully provided by the previous team (`mpu6050_to_serial`) which basically allows our external processor to extract the readings from the sensor through I2C protocol. Then sends it to the laptop by cable through serial communication.

- Second issue started to appear as the sensor started to behave awkwardly and noisily:
 - Disconnecting suddenly at random times
 - Changing the axis of rotation randomly causing the robot to penetrate the ground level on the real-time simulation using RVIZ.

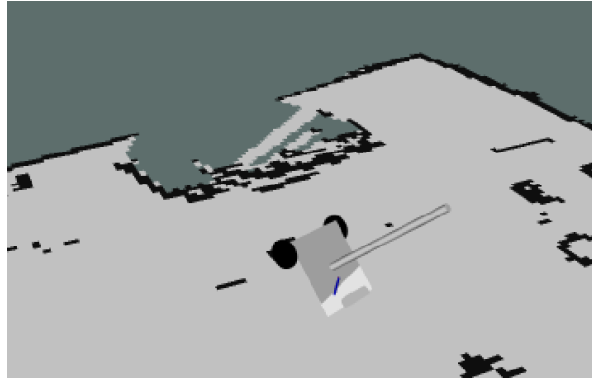


Figure 48 wrong Axis of rotation

As almost the same issues faced most of our colleagues in different teams we decided to think out of the Box and upgrade the sensor with an already existing idle one in everyone's pocket.

Using Internal IMU sensor of a smart phone:

after much searching, we came up to an outdated android application that sends the readings of smartphone internal sensor as ROS msgs containing these readings in a form of a plain quaternion msgs through Wi-Fi connection.

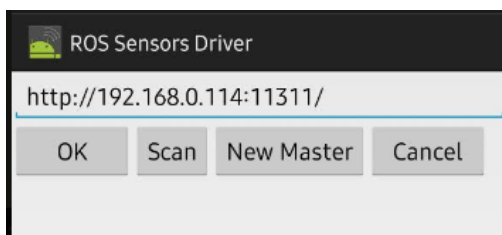


Figure 50 Mobile application

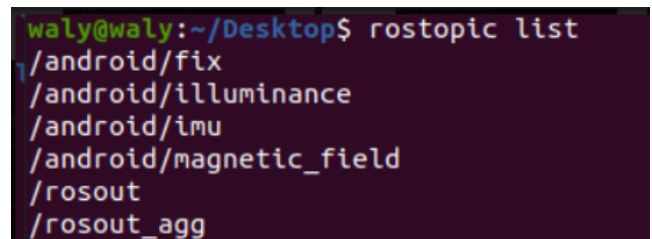


Figure 49 resulted Topics.

All what was remaining is to republish these readings with the needed transformation frames.

(imu_base and imu_link) for the rest of the system to be integrated with.

We manage to get that done by modifying the .cpp file already existed in the (mpu6050_to_serial) by getting the quaternion msgs through a subscriber we created instead of the serial communication.

Screenshots from the code:

```
ros::init(argc, argv, "android_spy_node");

ros::NodeHandle node;

node.param<std::string>("tf_parent_frame_id", tf_parent_frame_id, "imu_base"); //imu_base
node.param<std::string>("tf_frame_id", tf_frame_id, "imu_link");
node.param<std::string>("frame_id", frame_id, "imu_link");
node.param<double>("time_offset_in_seconds", time_offset_in_seconds, 0.0);
node.param<bool>("broadcast_tf", broadcast_tf, true);

ros::NodeHandle nh("imu");
ros::Publisher imu_pub = nh.advertise<sensor_msgs::Imu>("data", 50);

ros::Subscriber sub = node.subscribe("android/imu", 1000, magdyCallback);

ros::ServiceServer service = nh.advertiseService("set_zero_orientation", set_zero_orientation);

ros::Rate r(200); // 200 hz //30
sensor_msgs::Imu imu;

void AndroidCallback(const sensor_msgs::Imu::ConstPtr& msg)
{
    ox = msg->orientation.x ;
    oy = msg->orientation.y;
    oz = msg->orientation.z ;
    ow = msg->orientation.w ;

    ax = msg->angular_velocity.x ;
    ay = msg->angular_velocity.y;
    az = msg->angular_velocity.z ;

    lx = msg->linear_acceleration.x;
    ly = msg->linear_acceleration.y;
    lz = msg->linear_acceleration.z;
}
```

Figure 51 script that handles the mobile readings

This resulted in a very good behavior as the URDF rotated synchronously with the rotation of the phone.

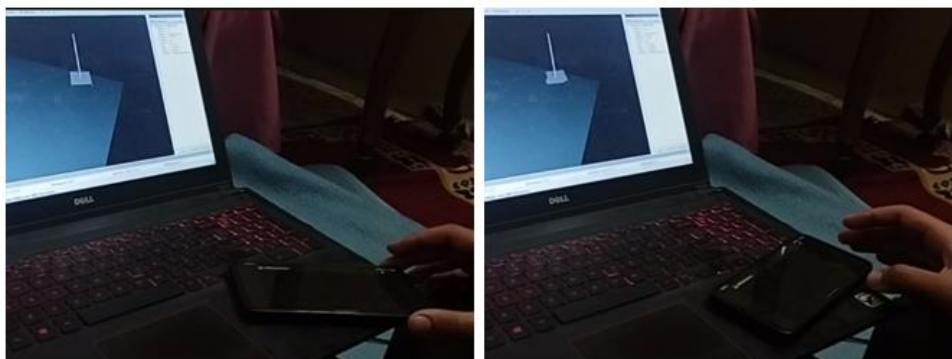


Figure 52 IMU mobile trial

We even added a condition to eliminate all possible rotations except for those around the z-axis.

Unfortunately, this path came to an end as we faced a problem we couldn't solve as we couldn't reference our readings properly and that lead to accumulative error in the readings (the MPU6050 readings were referenced to the magnetic field of the earth) trying to create a similar reference would often result in unconnected transformation tree and multiple RVIZ errors.

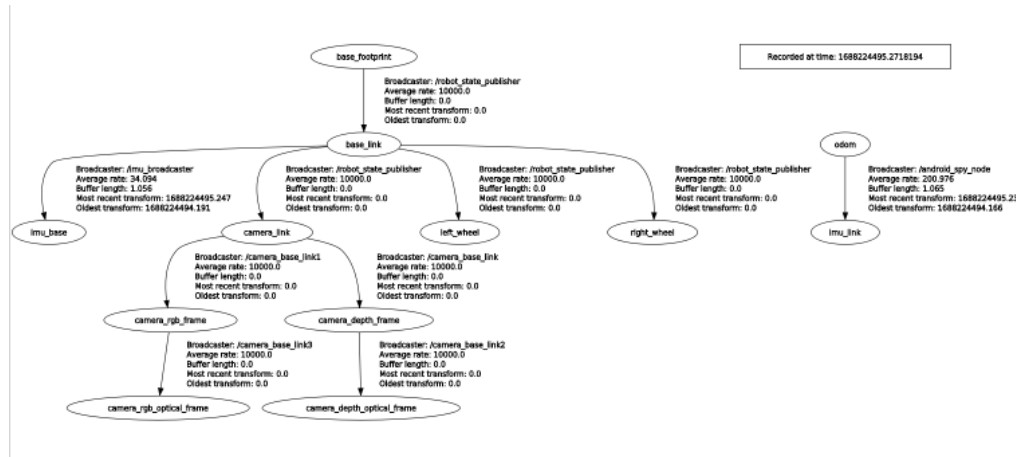


Figure 53 wrong resulted tf tree

The development of this part took much time and effort as we had no reference to follow. We tried to create as much facilities as we could.

Although this came to a dead end, it's still a widely open field for the next team to finish it as we already got close.

We then decided to return to the regular module after even calibrating it more. It worked fine but we were left with the problem of the sudden disconnection (after an average of 2 mins of operating time)

The unstable VCC connection was our main suspect, so we decided to design a PCB for the module connection with the Arduino.

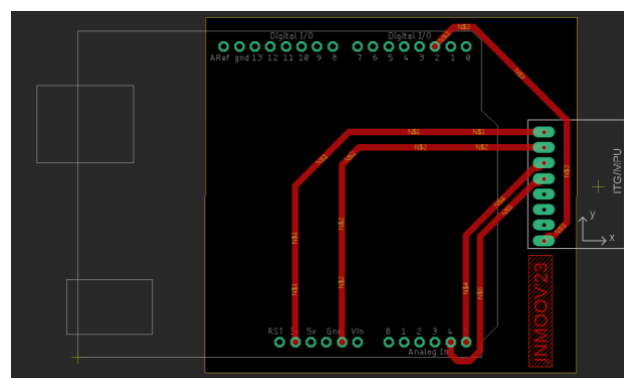


Figure 54 IMU PCB layout

A more stable connection would surely lead to a better performance.

Applying Localization on our robot:

Our robot uses 2 DC motors for travelling, they are both connected to the Cytron Dual Channel motor driver.

An Arduino Mega is responsible for sending the signals to operate both of the motors that's why an Arduino code is uploaded on it subscribing on the “cmd_vel” topic then manipulate the Twist msgs to generate the required PWM signals for both of the motors through a Proportional controller to assure the accuracy in the movement of the robot.

As the robot moves both the encoders and the IMU generate signals and readings, giving information about the location of the robot. Fusing this information would lead to a more accurate estimation of the current position of the robot. This was achieved using the extended Kalman filter ROS package.

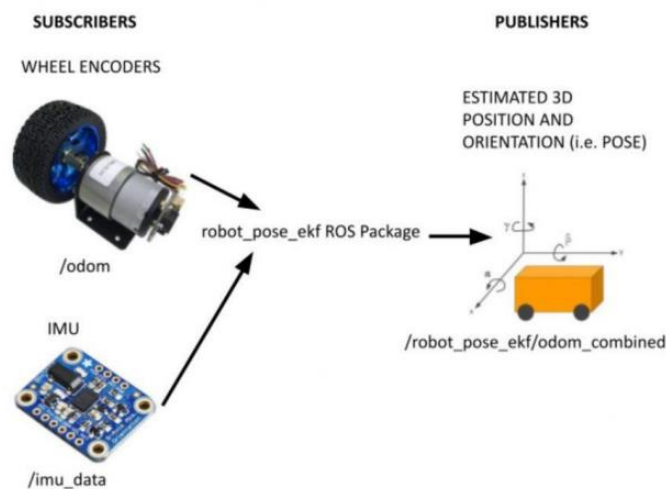


Figure 55 Robot Pose EKF

The ROS Robot Pose EKF package is a software module that estimates a robot's position and orientation based on sensor data. It fuses data from multiple sources (IMU and encoders in our case) to provide a more accurate estimate and compensates for sensor latency. It's commonly used in robotics applications that require precise positioning and is open-source with extensive documentation and support. Inputs are sensor data and the initial pose estimate, and the output is the filtered pose estimate.

4.1.4 Mapping:

In autonomous systems, mapping refers to the process of creating a representation of the environment in which the system operates. This can include creating a map of physical features such as buildings, roads, and terrain, as well as other relevant information such as traffic patterns, weather conditions, and potential obstacles.

It enables autonomous robots to navigate and operate effectively in their environment. By having a detailed and accurate map of the environment, autonomous systems can plan their movements and actions more efficiently, avoid obstacles, and adapt to changes in their surroundings.

RTAB mapping technique was used by us to perform the mapping task.

RTAB map:

RTAB_map is a visual SLAM which uses loop closure detection algorithm based on a memory management mechanism. It is implemented on ROS with a main node called “rtabmap”. The data inputs for node are odometry data from sensors fusion, image, depth image, camera info from the Kinect and output the map.

Kinect calibration:

we tried to calibrate our Kinect camera (the IR sensor and the RGB levels) by using Four corners checkboard paper.



Figure 56 Kinect calibration

We moved the checkboard in front of the camera to the far right and to the far left and up and down to calibrate the Kinect.

Calibration commands:

```
“roslaunch camera_calibration cameracalibrator.py image:=/camera/rgb/image_raw  
camera:=/camera/rgb --size 8x6 --square 0.035”
```

the previous command starts the calibration process for the rgb of the Kinect using a checker paper.

, noting that the dimensions of the squares and the number of the squares are provided in the command.

same for the following command except it's for calibrating the IR sensor of the Kinect.

```
“roslaunch camera_calibration cameracalibrator.py image:=/camera/ir/image_raw  
camera:=/camera/ir --size 8x6 --square 0.035”
```

Applying Mapping on our robot:

We launch the “openni” package which converts Kinect readings into depth images. We then launch the “depthimage_to_laserscan” package which uses the resulted RGB-D images to create a “scan” message as what a Lidar would do.

All the localization applications we explained at the previous section should be operating for the robot to localize itself properly in the map.

We launch the RTABmap package which subscribes on the scan and the Odom. Thus, generating the map with the proper dimensions.



Figure 57 Mapping Process

Using the “rqt_steering” node to manipulate the robot by publishing Twist messages on the “cmd_vel” topic. we move the robot until reach the map closure.

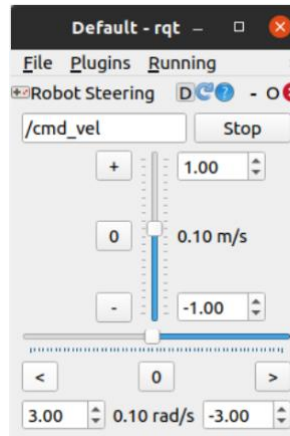


Figure 58 rqt_steering node interface

4.1.5 Navigation and path planning:

Navigation in autonomous systems refers to the process by which a robot moves from one location to another in its environment. Navigation involves a combination of perception, decision-making, and control algorithms that allow the robot to plan and execute a path through the environment while avoiding obstacles and achieving its goals.

Path planning:

By its robotics definition, path planning is the process you use to construct a path from a starting point to an ending point given a full, partial, or dynamic map. Essentially, you are calculating out a path on a particular terrain that is the shortest or fastest for a robot to navigate while also avoiding obstacles.

When you think of path planning you are answering questions like ‘which direction is your robot moving in?’, ‘where is the next step?’, and ‘how can I reach the destination with this barrier in my way?’

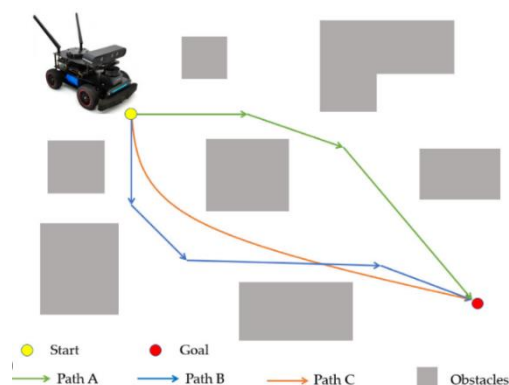


Figure 59 path planning

Evolution of path planning:

Mid 70s: Classical Robotics Path Planning:

- Task Based and Joint Based Trajectory planning.
- Constrained motion and environment.
- Exact models.

Mid 80s Reactive Paradigm:

- No model required.
- Highly relied upon sensory data.

Since 90: Hybrids Paradigm:

- Model-based at higher levels.
- Reactive at lower levels.

Since 2000s: Probabilistic Robotics:

- Seamless integration of models and sensors.
- Inaccurate models and inaccurate sensors.

Any path planning algorithm can be visualized as:

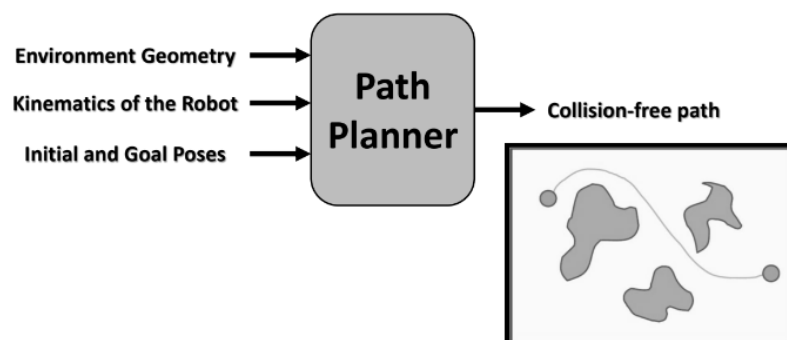


Figure 60 Path planner inputs and outputs

Optimization Approaches:

To be able to find the optimal path, optimization approaches are used, which can be classified as follows:

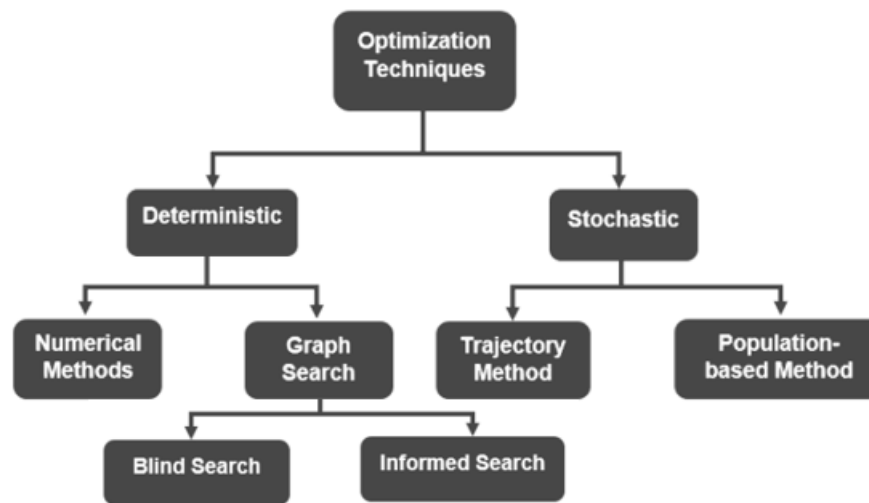


Figure 61 Optimization techniques

In deterministic models, the output of the model is fully determined by the parameter values and the initial conditions. Stochastic models possess some inherent randomness. The same set of parameter values and initial conditions will lead to an ensemble of different outputs. While in deterministic model, if have the same parameters values and initial condition it will always produce the output.

There are two techniques in Graph Search Methods. First Blind Search is to find the path without depending on any information of the position of the goal like Breadth first, Depth First and Dijkstra. Second informed search which the path is produced depending on position of the goal like Beam search, hill climbing and A*.

Every algorithm is evaluated on its speed, completeness, optimality, and feasibility of solution.

Method	Advantages	Disadvantages
Dijkstra	Optimal Shortest Path	Slow, no clearance guarantee.
A*	Fast with good heuristic, the result path is close to optimal.	The result path tends to stay close to obstacles.
RRT	Fast	No shortest or Clearance guarantee
Potential Field	Path Clearance is guaranteed.	Have local optimal.
Voronoi Diagram	Path Clearance is guaranteed.	The result path is smooth or does not satisfy vehicle dynamic constraints

Table 4 Comparison between path planning technique

For navigation to be performed well, it needs some inputs to generate an optimum path: Map topology - Robot initial pose - Obstacle status - Goal point pose.

An autonomous robot must use its sensory data and fractional information about its surroundings to reach its desired destination or set of locations in an efficient and reliable manner. This involves integrating tools from previous sections (Localization and mapping) to solve the navigation problem. By using these tools together, the robot can plan a safe and efficient path to its destination and execute the path while avoiding obstacles and adjusting its movements based on sensory feedback. The goal of this section is to explore how these different tools can be combined to enable navigation in

Navigation in ROS:

When we relate navigation to ROS, we found that ROS offers what is called navigation stack. The Navigation Stack is simple on a conceptual level. It takes in information from odometry, sensor streams and outputs velocity commands to send to a mobile base.

Navigation stack has two main types of planners, one is called Global Planner and the other is called Local Planner

Global Planner:

In navigation, a global planner is a component of the path planning module that generates an overall plan to reach the robot's final destination from its current location. The global planner takes into account the robot's starting location, the destination, and any obstacles or constraints in the environment, such as walls, furniture, or other objects.

From the multiple options available at ROS for the global planner we worked with “NavfnROS” planner

NavfnROS

navfn provides a fast interpolated navigation function that can be used to create plans for a mobile base. The planner assumes a circular robot and operates on a costmap to find a minimum cost plan from a start point to an end point in a grid. The navigation function is computed with Dijkstra's algorithm.

Local Planner:

A local planner is a component of the path planning module that generates a detailed trajectory or path for the robot to follow between two waypoints. It receives input from the global planner, which provides a high-level plan for the robot's overall path through the environment. The local planner uses this plan to generate a sequence of intermediate waypoints that the robot must reach to follow the overall path. It then generates a detailed trajectory that connects each pair of waypoints, taking into account the robot's speed, acceleration, and turning radius, as well as any obstacles or constraints in the environment.

The local planner used with our project was “DwaPlannerROS”.

DWA local planner:

Dynamic Window Approach to local robot navigation on a plane. Given a global plan to follow and a costmap, the local planner produces velocity commands to send to a mobile base. This package supports any robot whose footprint can be represented as a convex polygon or circle, and exposes its configuration as ROS parameters that can be set in a launch file

move_base:

The move_base package provides an implementation of an action that, given a goal in the world, will attempt to reach it with a mobile base. The move_base node links together a global and local planner to accomplish its global navigation task.

This package provides the **move_base ROS Node** which is a major component of the navigation stack.

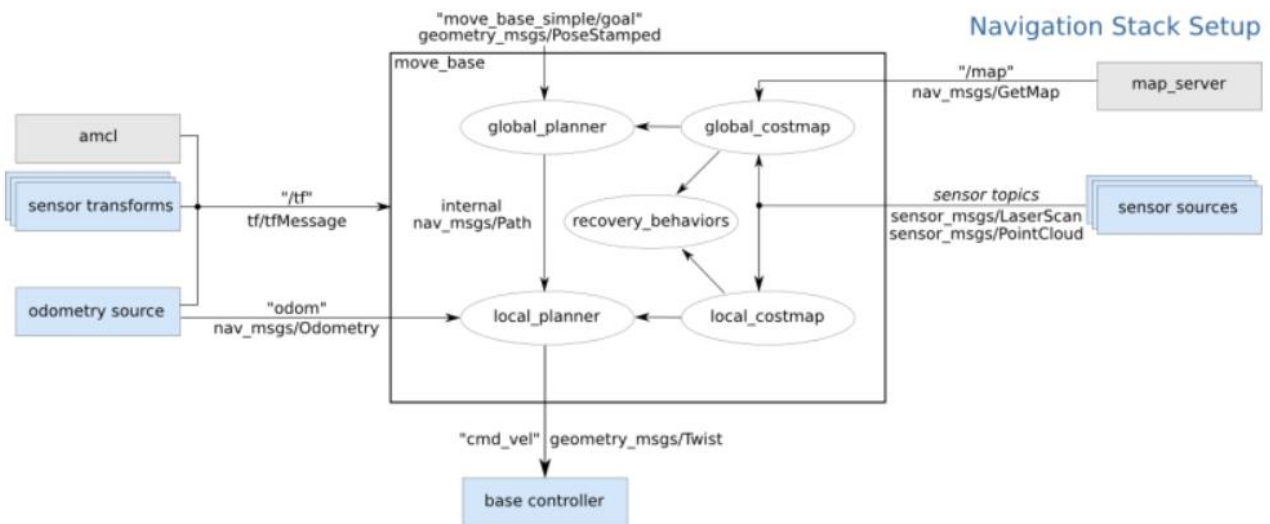


Figure 62 ROS navigation Stack

move_base will send velocity commands for the robot to follow the desired path to reach the goal.

```
<!-- Move Base Node -->
<!-- Subscribe: /move_base_simple/goal -->
<!-- Publish: /cmd_vel -->
<node pkg="move_base" type="move_base" respawn="false" name="move_base" output="screen">

  <roscparam file="$(find navros_pkg)/param/costmap_common_params.yaml" command="load" ns="global_costmap" />
  <roscparam file="$(find navros_pkg)/param/costmap_common_params.yaml" command="load" ns="local_costmap" />
  <roscparam file="$(find navros_pkg)/param/local_costmap_params.yaml" command="load" />
  <roscparam file="$(find navros_pkg)/param/global_costmap_params.yaml" command="load" />
  <roscparam file="$(find navros_pkg)/param/dwa_local_planner_params.yaml" command="load" />
  <roscparam file="$(find navros_pkg)/param/move_base_params.yaml" command="load" />
  <roscparam file="$(find navros_pkg)/param/navfn_global_planner_params.yaml" command="load" />

</node>
```

Figure 63 move base configurations.

Costmap:

A costmap is a grid map where each cell is assigned a specific value or cost: higher costs indicate a smaller distance between the robot and an obstacle, and It is based on the occupancy grid map of the environment and user specified inflation radius.

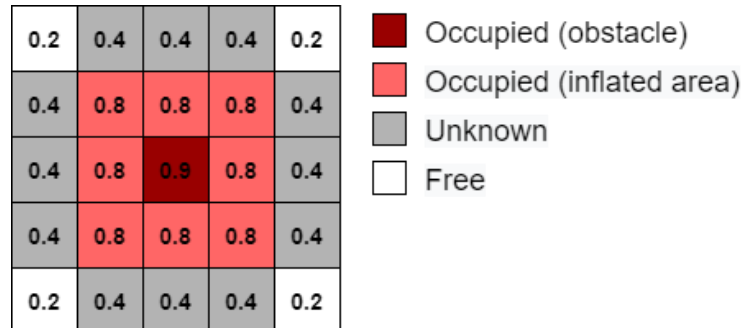


Figure 64 Costmap

There are two types of costmaps in ROS: Global costmap is used for global navigation and Local costmap is used for local navigation.

In each cycle:

Sensor data comes in – Marking and clearing operations are performed in the underlying occupancy structure of the costmap – This structure is projected into the costmap where the appropriate cost values are assigned.

Costmap Layers:

1. Static Map Layer: The static map layer represents a largely unchanging portion of the costmap, like those generated by SLAM.
2. Obstacle Map Layer: The obstacle layer tracks the obstacles as read by the sensor data. The Obstacle Costmap Plugin marks and raytraces obstacles in two dimensions, while the Voxel Costmap Plugin does so in three dimensions.
3. Inflation Layer: The inflation layer is an optimization that adds new values around lethal obstacles (i.e., inflates the obstacles) in order to make the costmap represent the configuration space of the robot.

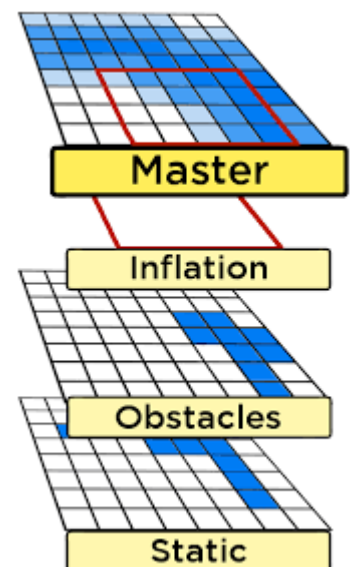


Figure 65 Costmap layers

4.2 DETECTION SYSTEMS

4.2.1 Sound detection and location

The robot is provided with a localization module that analyses observed sound signals and determines the positions of all currently active sound sources.

The required component to perform the sound localization function is: ky-037 sound detector sensor. This sensor detects any sound around it and produces corresponding analog and digital signals.

By placing 3 or more sensors in different parts of the head for example, the two ears and the bottom, we can determine the nearest location to the sound.

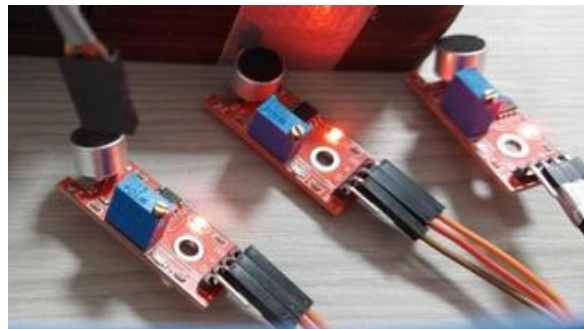


Figure 66 microphone sensor

Algorithm:

The sound localization mechanism is similar to the design of our hearing system since the sound in the air has a certain velocity 343 m/s.

This means that if the sound source is on our left, then the sound wave will reach to our left ear before it reaches to the right ear, and if it was in front of the 2 ears, then the sound will reach to both ears at the same time, and if it was between the front and left positions, then the sound wave will also reach to the left ear first but it will take less time to reach to the right ear, which our brain can sense this difference in time and detect that the source is in 45 degrees from the left direction, and vice versa.

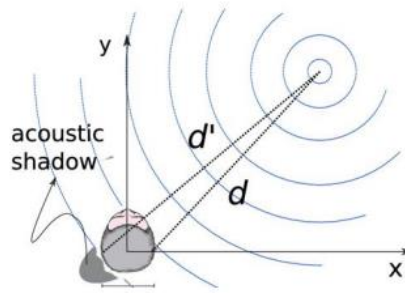


Figure 67 Sound localization mechanism

For horizontal detection (right, left, front) in the head:

1. Left sensor (left shoulder)
2. Front sensor (front of chest)
3. Right sensor (right shoulder)

as the distance between the sensors increase, the error rate will decrease as the time of sound signal reaching the far sensors after it reaches the nearest sensor will increase

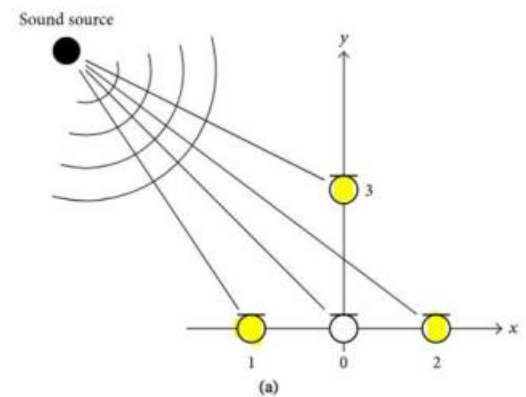


Figure 68 sensors positioning

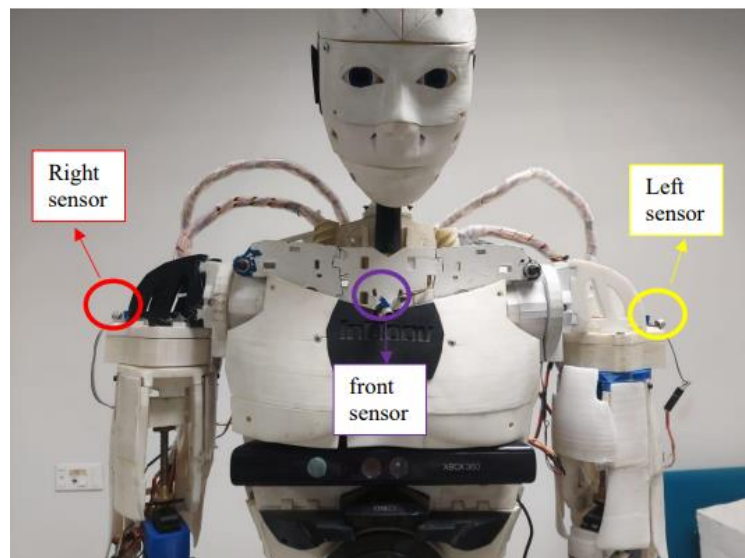


Figure 69 sensors locations on the robot

4.2.2 Object Recognition

Object detection is a computer vision task that involves identifying and locating objects in images or videos. It is an important part of many applications, such as surveillance, self-driving cars, or robotics. Object detection algorithms can be divided into two main categories: single-shot detectors and two-stage detectors.

One and two stage detectors

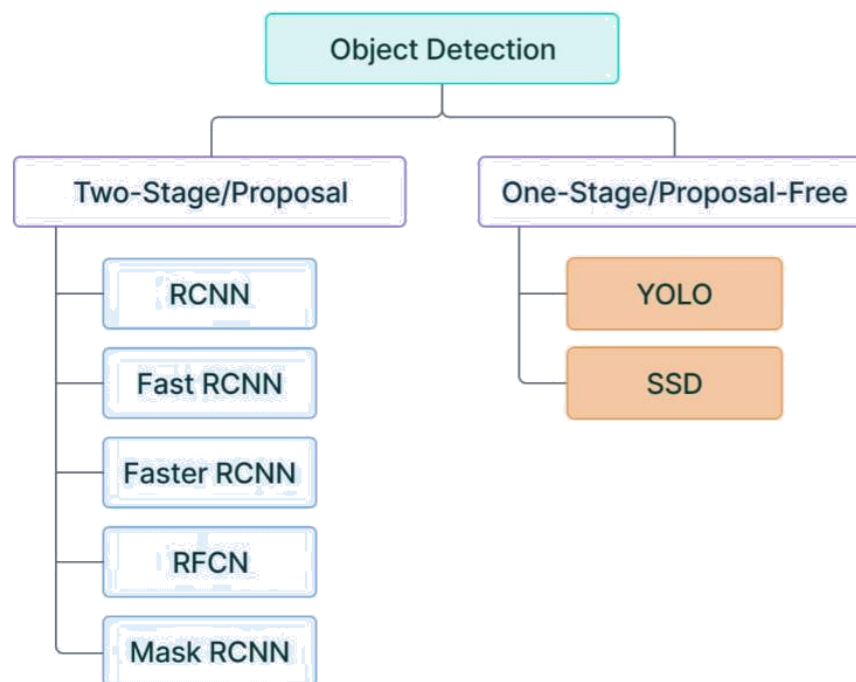


Figure 70 Object detection classification

Single-shot object detection:

Single-shot object detection uses a single pass of the input image to make predictions about the presence and location of objects in the image. It processes an entire image in a single pass, making them computationally efficient.

However, single-shot object detection is generally less accurate than other methods, and it's less effective in detecting small objects. Such algorithms can be used to detect objects in real time in resource-constrained environments.

YOLO is a single-shot detector that uses a fully convolutional neural network (CNN) to process an image.

Two-shot object detection:

Two-shot object detection uses two passes of the input image to make predictions about the presence and location of objects. The first pass is used to generate a set of proposals or potential object locations, and the second pass is used to refine these proposals and make final predictions. This approach is more accurate than single-shot object detection but is also more computationally expensive.

Overall, the choice between single-shot and two-shot object detection depends on the specific requirements and constraints of the application.

Generally, single-shot object detection is better suited for real-time applications, while two-shot object detection is better for applications where accuracy is more important.

So, we chose to work on the YOLO single-shot object detection.

Object detection models performance evaluation:

To determine and compare the predictive performance of different object detection models, we need standard quantitative metrics.

Mean Average Precision (mAP):

- **is commonly used to analyze the performance of object detection and segmentation systems**, many object detection algorithms, such as Faster R-CNN, MobileNet SSD, and YOLO use mAP to evaluate their models.
- **The mean of average precision (AP) values is calculated over recall values from 0 to 1.**
- **mAP formula is based on the following sub metrics:**
 - **Confusion Matrix,**
 - **Intersection over Union (IoU),**
 - **Recall**
 - **Precision**

Confusion Matrix:

To create a confusion matrix, we need four attributes:

- True Positives (TP): The model predicted a label and matches correctly as per ground truth.
- True Negatives (TN): The model does not predict the label and is not a part of the ground truth.
- False Positives (FP): The model predicted a label, but it is not a part of the ground truth.
- False Negatives (FN): The model does not predict a label, but it is part of the ground truth.

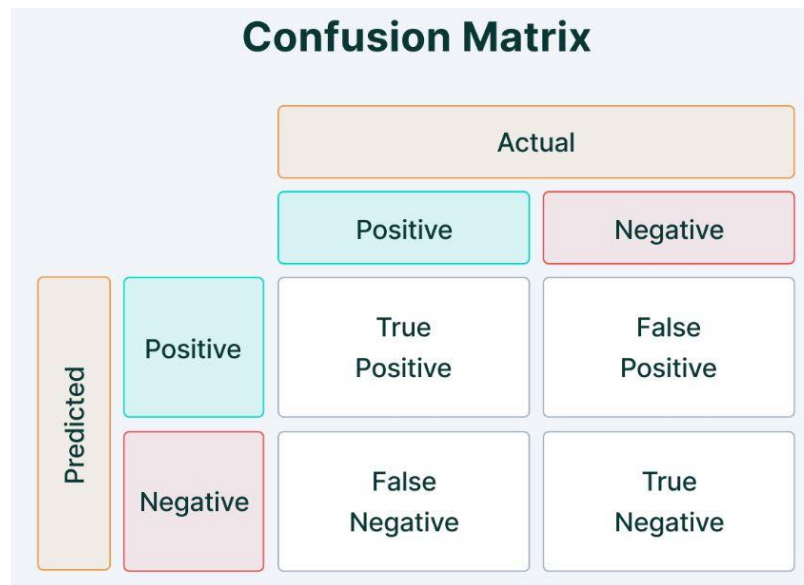


Figure 71 Confusion Matrix

Intersection over Union (IoU):

- Intersection over Union indicates the overlap of the predicted bounding box coordinates to the ground truth box.
- Higher IoU indicates the predicted bounding box coordinates closely resembles the ground truth box coordinates.

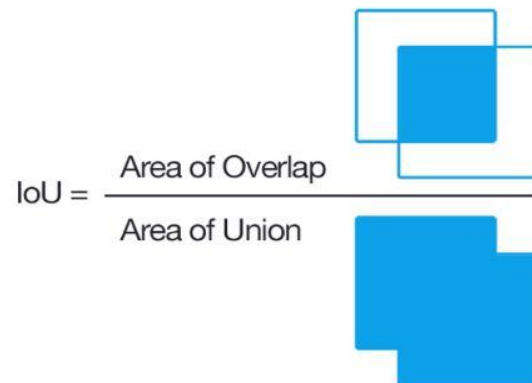
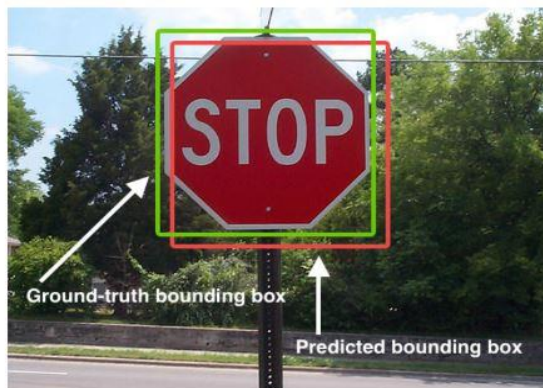


Figure 72 Intersection over union

Precision:

Precision measures how well you can find true positives (TP) out of all positive predictions. (TP+FP).

$$Precision = \frac{TP}{TP + FP}$$

Recall:

Recall measures how well you can find true positives (TP) out of all predictions (TP+FN).

$$Recall = \frac{TP}{TP + FN}$$

The steps to calculate the AP:

1. Generate the prediction scores using the model.
2. Convert the prediction scores to class labels.
3. Calculate the confusion matrix—TP, FP, TN, FN.
4. Calculate the precision and recall metrics.
5. Calculate the area under the precision-recall curve.
6. Measure the average precision.
- 7.

The mAP is calculated by finding Average Precision (AP) for each class and then averaging over a number of classes.

The mAP incorporates the trade-off between precision and recall and considers both false positives (FP) and false negatives (FN). This property makes mAP a suitable metric for most detection applications.

COCO mAP:

According to the COCO 2017 challenge evaluation guidelines, the mAP was calculated by averaging the AP over 80 object classes AND all 10 IoU thresholds from 0.5 to 0.95 with a step size of 0.05.

The primary challenge metric in COCO 2017 challenge is calculated as follows:

- AP is calculated for the IoU threshold of 0.5 for each class.
- Calculate the precision at every recall value (0 to 1 with a step size of 0.01), then it is repeated for IoU thresholds of 0.55, 0.60, ..., 0.95.
- Average is taken over all the 80 classes and all the 10 thresholds.

	backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
<i>Two-stage methods</i>							
Faster R-CNN+++ [3]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [6]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [4]	Inception-ResNet-v2 [19]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [18]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
<i>One-stage methods</i>							
YOLOv2 [13]	DarkNet-19 [13]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [9, 2]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [2]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [7]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [7]	ResNeXt-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

Figure 73 COCO for YOLOv3

YOLO:

We used the YOLO ROS: Real-Time object detection for ROS technique to apply the object detection feature in our system.

Overview:

- You Only Look Once (YOLO) proposes using an end-to-end neural network that makes predictions of bounding boxes and class probabilities all at once. It differs from the approach taken by previous object detection algorithms, which repurposed classifiers to perform detection.
- This is a ROS package developed for object detection in camera images. You only look once (YOLO) is a state-of-the-art, real-time object detection system.
- In the following ROS package, you are able to use YOLO (V3) on GPU and CPU. The pre-trained model of the convolutional neural network is able to detect pre-trained classes including the data set from VOC and COCO, or you can also create a network with your own detection objects.
- While algorithms like Faster RCNN work by detecting possible regions of interest using the Region Proposal Network and then performing recognition on those regions separately, YOLO performs all of its predictions with the help of a single fully connected layer.
- Methods that use Region Proposal Networks perform multiple iterations for the same image, while YOLO gets away with a single iteration.
- It was first introduced by Joseph Redmon et al. in 2015.
 - Based on the Pascal VOC 2012 dataset, YOLO can detect the 20 Pascal object classes.
 - Based on the COCO dataset, YOLO can detect the 80 COCO object classes.

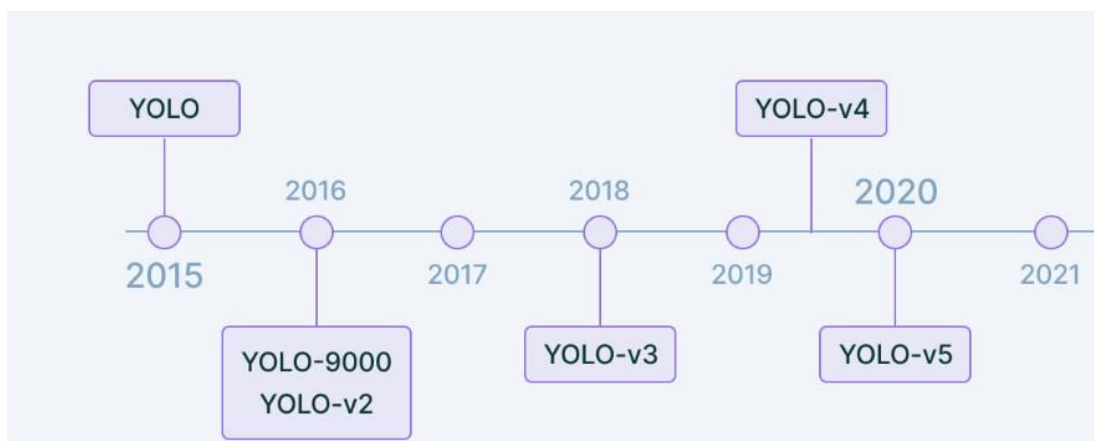


Figure 74 YOLO timeline

Comparison to Other Detectors:

YOLOv3 is extremely fast and accurate. In mAP measured at .5 IOU YOLOv3 is on par with Focal Loss but about **4x faster**. Moreover, you can easily trade off between speed and accuracy simply by changing the size of the model, no retraining required!

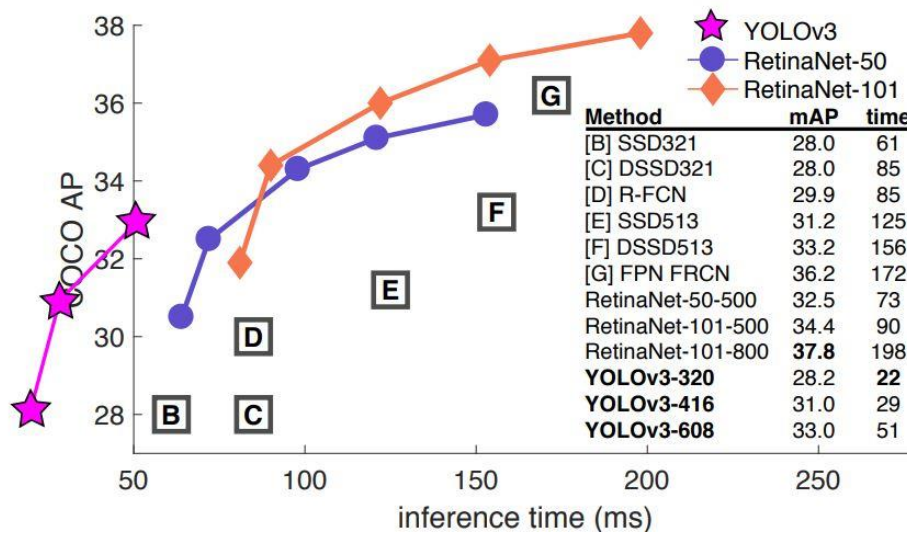


Figure 75 YOLO vs other detectors

Performance on the COCO Dataset

Model	Train	Test	mAP	FLOPS	FPS
SSD300	COCO trainval	test-dev	41.2	-	46
SSD500	COCO trainval	test-dev	46.5	-	19
YOLOv2 608x608	COCO trainval	test-dev	48.1	62.94 Bn	40
Tiny YOLO	COCO trainval	test-dev	23.7	5.41 Bn	244
SSD321	COCO trainval	test-dev	45.4	-	16
DSSD321	COCO trainval	test-dev	46.1	-	12
R-FCN	COCO trainval	test-dev	51.9	-	12
SSD513	COCO trainval	test-dev	50.4	-	8
DSSD513	COCO trainval	test-dev	53.3	-	6
FPN FRCN	COCO trainval	test-dev	59.1	-	6
Retinanet-50-500	COCO trainval	test-dev	50.9	-	14
Retinanet-101-500	COCO trainval	test-dev	53.1	-	11
Retinanet-101-800	COCO trainval	test-dev	57.5	-	5
YOLOv3-320	COCO trainval	test-dev	51.5	38.97 Bn	45
YOLOv3-416	COCO trainval	test-dev	55.3	65.86 Bn	35
YOLOv3-608	COCO trainval	test-dev	57.9	140.69 Bn	20
YOLOv3-tiny	COCO trainval	test-dev	33.1	5.56 Bn	220
YOLOv3-spp	COCO trainval	test-dev	60.6	141.45 Bn	20

Figure 76 YOLO v3 Performance

How does YOLO work? (YOLO Architecture):

The YOLO algorithm takes an image as input and then uses a simple deep convolutional neural network to detect objects in the image. The architecture of the CNN model that forms the backbone of YOLO is shown below.

So now we are going to explain how CNN networks work to understand YOLO idea of working:

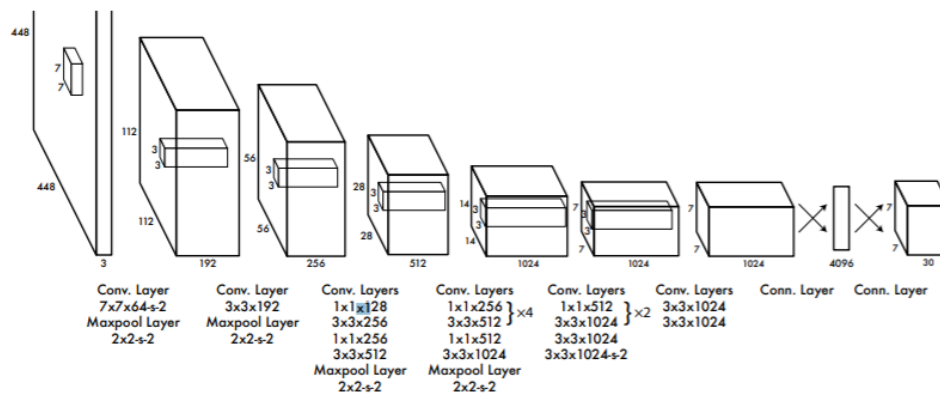


Figure 77 CNN layers

CNN (Convolutional Neural Network):

- CNNs are a class of Deep Neural Networks that can recognize and classify particular features from images and are widely used for analyzing visual images.
- Their applications range from image and video recognition, image classification, medical image analysis, computer vision and natural language processing.
- CNN has high accuracy, and because of the same, it is useful in image recognition.
- The term ‘Convolution’ in CNN denotes the mathematical function of convolution which is a special kind of linear operation where in two functions are multiplied to produce a third function which expresses how the shape of one function is modified by the other. In simple terms, two images which can be represented as matrices are multiplied to give an output that is used to extract features from the image.

CNN typically has three layers: a convolutional layer, a pooling layer, and a fully connected layer.

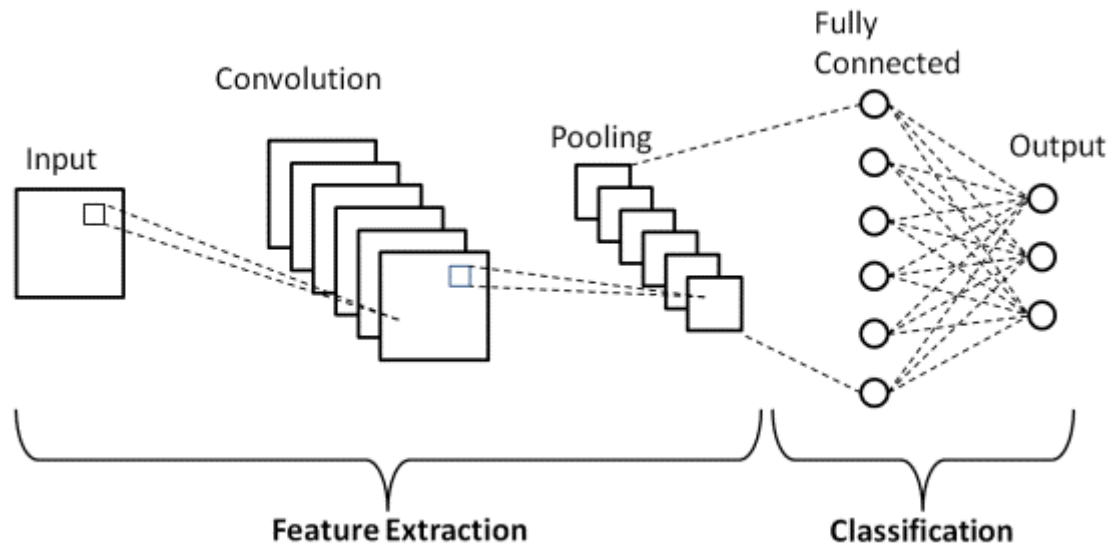


Figure 78 CNN extraction and classification

There are two main parts to a CNN architecture:

1) Feature extraction:

- A convolution tool that separates and identifies the various features of the image for analysis in a process called **Feature Extraction**.
- The network of feature extraction consists of many pairs of convolutional or pooling layers.

2) Classification:

A fully connected layer that utilizes the output from the convolution process and predicts the class of the image based on the features extracted in previous stages.

There are three types of layers that make up the CNN which are the convolutional layers, pooling layers, and fully connected (FC) layers.

When these layers are stacked, a CNN architecture will be formed. In addition to these three layers, there are two more important parameters which are the dropout layer and the activation function.

1. Convolutional Layer:

This layer is the first layer that is used to extract the various features from the input images. In this layer, the mathematical operation of convolution is performed between the input image and a filter of a particular size $M \times M$. By sliding the filter over the input image, the dot product is taken between the filter and the parts of the input image with respect to the size of the filter ($M \times M$).

The output is termed as the Feature map which gives us information about the image such as the corners and edges. Later, this feature map is fed to other layers to learn several other features of the input image.

The convolution layer in CNN passes the result to the next layer once applying the convolution operation in the input.

2. Pooling Layer:

In most cases, a Convolutional Layer is followed by a Pooling Layer. The primary aim of this layer is to decrease the size of the convolved feature map to reduce the computational costs. This is performed by decreasing the connections between layers and independently operates on each feature map. Depending upon the method used, there are several types of Pooling operations. It basically summarizes the features generated by a convolution layer.

In Max Pooling, the largest element is taken from the feature map. Average Pooling calculates the average of the elements in a predefined sized Image section. The total sum of the elements in the predefined section is computed in Sum Pooling. The Pooling Layer usually serves as a bridge between the Convolutional Layer and the FC Layer.

This CNN model generalizes the features extracted by the convolution layer and helps the networks to recognize the features independently. With the help of this, the computations are also reduced in a network.

3. Fully Connected Layer:

The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers. These layers are usually placed before the output layer and form the last few layers of CNN Architecture.

In this, the input image from the previous layers is flattened and fed to the FC layer. The flattened vector then undergoes a few more FC layers where the mathematical functions operations usually take place. In this stage, the classification process begins to take place. The reason two layers are connected is that two fully connected layers will perform better than a single connected layer. These layers in CNN reduce human supervision.

4. Dropout:

Usually, when all the features are connected to the FC layer, it can cause overfitting in the training dataset. Overfitting occurs when a particular model works so well on the training data causing a negative impact in the model's performance when used on a new data.

To overcome this problem, a dropout layer is utilized wherein a few neurons are dropped from the neural network during training process resulting in reduced size of the model. On passing a dropout of 0.3, 30% of the nodes are dropped out randomly from the neural network.

Dropout results in improving the performance of a machine learning model as it prevents overfitting by making the network simpler. It drops neurons from the neural networks during training.

5. Activation Functions:

Finally, one of the most important parameters of the CNN model is the activation function. They are used to learn and approximate any kind of continuous and complex relationship between variables of the network. In simple words, it decides which information of the model should fire in the forward direction and which ones should not at the end of the network. Now we are going to explain how YOLO depends on the CNN to make object detection process:

The YOLO Detection System:

- Processing images with YOLO is simple and straightforward:
 - (1) Resizes the input image to 448×448 .
 - (2) Runs a single convolutional network on the image.
 - (3) Thresholds the resulting detections by the model's confidence.
- A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation.
- Our unified architecture is extremely fast. Our YOLO model processes images in real-time at 45 frames per second. A smaller version of the network, Fast YOLO processes 155 frames per second while still achieving double the mAP of other real-time detectors.
- YOLO unifies the separate components of object detection into a single neural network. Our network uses features from the entire image to predict each bounding box. It also predicts all bounding boxes across all classes for an image simultaneously.
- This means our network reasons globally about the full image and all the objects in the image.
- The YOLO design enables end-to-end training and real time speeds while maintaining high average precision.
- YOLO is pretrained through convolutional layers on the ImageNet 1000-class competition dataset. For pretraining YOLO uses the first 20 convolutional layers followed by an average-pooling layer and a fully connected layer.
- Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

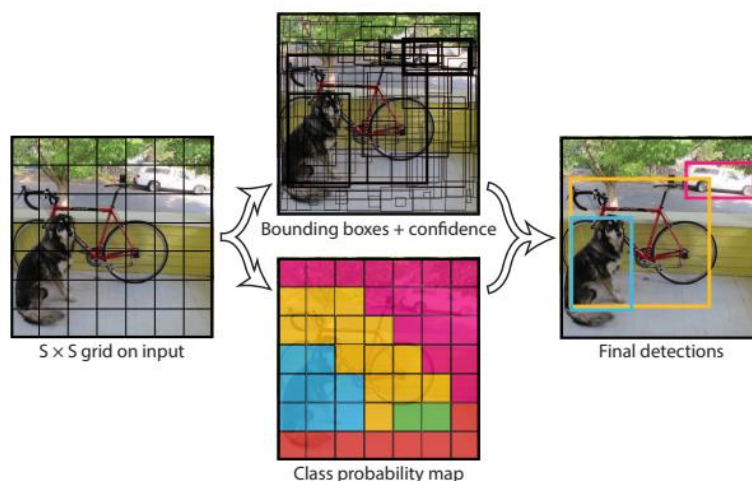


Figure 79 Detection steps

Limitations of YOLO:

- YOLO imposes strong spatial constraints on bounding box predictions since each grid cell only predicts two boxes and can only have one class.
- This spatial constraint limits the number of nearby objects that our model can predict, our model struggles with small objects that appear in groups, such as flocks of birds.
- Since our model learns to predict bounding boxes from data, it struggles to generalize objects in new or unusual aspect ratios or configurations.
- Our model also uses relatively coarse features for predicting bounding boxes since our architecture has multiple down sampling layers from the input image.



Figure 80 Detection in action

Why YOLO v3?

- YOLO v3 is the third version of the YOLO object detection algorithm. It was introduced in 2018 as an improvement over YOLO v2, aiming to increase the accuracy and speed of the algorithm.
- One of the main improvements in YOLO v3 is the use of a new CNN architecture called Darknet-53. Darknet-53 is a variant of the ResNet architecture and is designed specifically for object detection tasks. It has 53 convolutional layers and is able to achieve state-of-the-art results on various object detection benchmarks.
- Another improvement in YOLO v3 is anchor boxes with different scales and aspect ratios. In YOLO v2, the anchor boxes were all the same size, which limited the ability of the algorithm to detect objects of different sizes and shapes. In YOLO v3 the anchor boxes are scaled, and aspect ratios are varied to better match the size and shape of the objects being detected.
- YOLO v3 also introduces the concept of "feature pyramid networks" (FPN). FPNs are a CNN architecture used to detect objects at multiple scales. They construct a pyramid of feature maps, with each level of the pyramid being used to detect objects at a different scale. This helps to improve the detection performance on small objects, as the model is able to see the objects at multiple scales. In addition to these improvements, YOLO v3 can handle a wider range of object sizes and aspect ratios. It is also more accurate and stable than the previous versions of YOLO.

	Type	Filters	Size	Output
1x	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
2x	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
8x	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
8x	Residual			32 × 32
	Convolutional	512	3 × 3 / 2	16 × 16
	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
4x	Residual			16 × 16
	Convolutional	1024	3 × 3 / 2	8 × 8
	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
4x	Residual			8 × 8
	Residual			8 × 8
Avgpool		Global		
Connected		1000		

Figure 81: Darknet-53

The Bridge between Raw image and ROS image:

- To process raw camera images into useful inputs to vision algorithms as YOLO in our case so we use the CV_camera node.

CV_camera node:

- This node uses camera_info_manager for dealing with camera_info.
- The camera_info_manager provides a C++ class used by many camera drivers to manage the camera calibration data required by the ROS image pipeline.
- The image pipeline fills the gap between getting raw images from a camera driver and higher-level vision processing.
- The image_pipeline is designed to process raw camera images into useful inputs to vision algorithms: rectified mono/color images, stereo disparity images, and stereo point clouds.

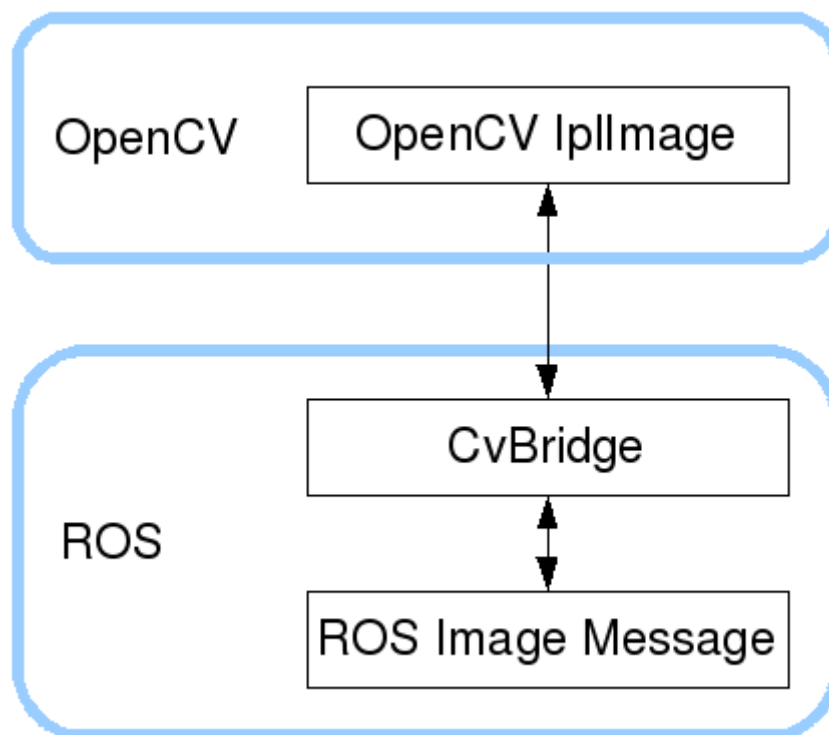


Figure 82 Detection in ROS

4.2.3 Sign recognition:

The sign detection algorithm resembles how the team got involved through understanding the system of InMoov and ROS tools.

This whole application was created from scratch using an open-source python file which Uses OpenCV library to detect the sign then it generates an array of 5 numbers corresponding to the sign shown, this file was established as a ROS node to publish this array using (Int23multiarray) from the standard messages.

ROS serial Arduino node was created to subscribe to a topic that gets the array from the recognition node then it sends the required signals to the motors to operate them.

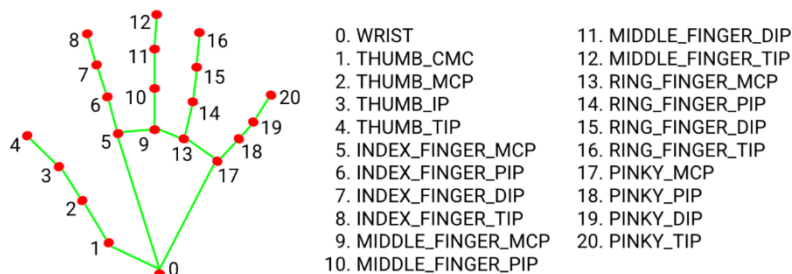


Figure 83 Sign recognition



Figure 84: Sign detection testing

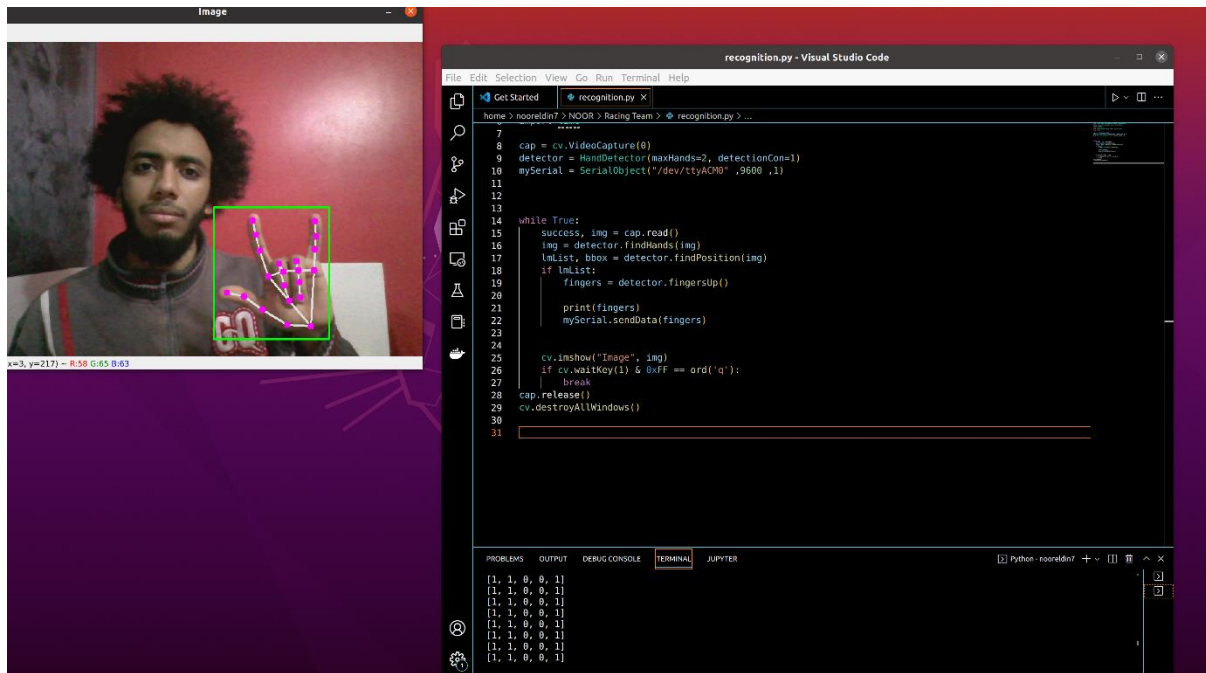


Figure 85: Sign detection testing

4.2.3 Speech Recognition

Speech-to-text technology has become increasingly popular in recent years due to its ability to transcribe spoken words into written text. This technology has numerous applications, including automated transcription services, virtual assistants, and customer service chatbots. However, the potential of speech-to-text technology is not limited to transcription alone. By using artificial intelligence (AI) algorithms to process the transcribed text, it is possible to generate answers and insights that can be used in a variety of applications. In this report, we will discuss how speech-to-text technology can be used in conjunction with AI to generate answers and insights, and how text-to-speech technology can be used to convert the data back into speech.



Figure 86

Speech-to-Text Technology:

Speech-to-text technology uses machine learning algorithms to transcribe spoken words into written text. The process involves breaking down the spoken words into individual sounds, identifying the words and phrases being used, and then converting these sounds into written text. This technology has been used for a variety of applications, including dictation software, automated transcription services, and customer service chatbots.



Figure 87 Speech to Text

Processing the Data:

Once the spoken words have been transcribed into written text, the data can be processed using AI algorithms. AI can be used to analyze the text, identify patterns, and extract key information. This can be done using techniques such as natural language processing (NLP) and machine learning. The results of the AI analysis can then be used to generate answers and insights that can be used in a variety of applications. We use it to generate responses accordingly.



Figure 88

Generating Answers:

After converting the speech to text, we use it to generate answers either by using a predefined response as hello and what is your name or using google bard's AI to generate answers for anything else.



Figure 89 Google Bard

Taking actions:

Then the actions are taken by moving or shaking hands etc. if it is told to do so. Else it will only respond to the question/command without moving.

Using Text-to-Speech Technology:

Once the data has been processed and answers have been generated, text-to-speech technology can be used to convert the written text back into speech. This technology has become increasingly sophisticated in recent years, allowing for more natural-sounding speech. Text-to-speech technology can be used in a variety of applications, including virtual assistants and automated telephone systems. We use it to give the response back to the user in a more comfortable way to easily communicate with the robot and to make it feel more like a humanoid that can interact with humans normally.



Figure 90

Conclusion:

Speech-to-text technology, combined with AI and text-to-speech technology, has the potential to revolutionize the way we communicate with machines. By transcribing spoken words into written text, processing the data using AI algorithms, generating answers and insights, and converting the data back into speech using text-to-speech technology, it is possible to create seamless and efficient communication systems that can be used in a variety of applications as we use it to make a humanoid robot that communicates easily with the user.

Speech recognition is the ability to convert the speech to text to be easily processed, we take the speech through a dedicated microphone connected to the laptop and using python we could convert the speech to text.

Then we process the text and send it using ROS to other nodes to use the commands/ text input that we got to take actions accordingly or determine the reply using predefined replies or using google bard's AI and convert it to speech using text to speech from google and save it as mp3 file to be then played through speakers connected to the laptop.

And here is a guide for the robot speech commands:

Speech	Reply	Action
السلام عليكم!	وعليكم السلام!	
Hello!	Hello, how are you?	
How are you?	I am fine, thank you!	
What is your name?	My name is INMOOV, and I am a humanoid robot.	
Can you help me?	Of course, yes, what can I do for you?	
Shake hands.	I will shake hands.	Publish: “shake hands”.
Move forward.	I will move forward.	Publish: “shake hands”.
Move backward.	I will move backward.	Publish: “back ward”.
Rotate.	I will Rotate.	Publish: “rotate”.
What can you see? / What do you see?	I see + “the output of object detection”	Receive the output of object detection and reads it
Anything else	Replies with the answer generated using google bard AI	Uses google bard’s AI to generate a reply
Close.	Goodbye!	Close the program.

Table 5 Speech Commands

4.3 ROBOT KINEMATICS

Robot kinematics applies geometry to the study of the movement of multi-degree of freedom kinematic chains that form the structure of robotic systems. The emphasis on geometry means that the links of the robot are modelled as rigid bodies and its joints are assumed to provide pure rotation or translation.

Robot kinematics studies the relationship between the dimensions and connectivity of kinematic chains and the position, velocity, and acceleration of each of the links in the robotic system, in order to plan and control movement and to compute actuator forces and torques. The relationship between mass and inertia properties, motion, and the associated forces and torques is studied as part of robot dynamics.

Forward kinematics:

Forward kinematics specifies the joint parameters and computes the configuration of the chain. For serial manipulators this is achieved by direct substitution of the joint parameters into the forward kinematics equations for the serial chain. For parallel manipulators substitution of the joint parameters into the kinematics equations requires solution of a set of polynomial constraints to determine the set of possible end-effector locations.

Inverse kinematics:

Inverse kinematics specifies the end-effector location and computes the associated joint angles. For serial manipulators this requires solution of a set of polynomials obtained from the kinematics equations and yields multiple configurations for the chain. The case of a general 6R serial manipulator (a serial chain with six revolute joints) yields sixteen different inverse kinematics solutions, which are solutions of a sixteenth-degree polynomial. For parallel manipulators, the specification of the end-effector location simplifies the kinematics equations, which yields formulas for the joint parameters.

Importance of robot kinematics for our project:

Since we are working on an autonomous Humanoid robot therefore, we have two hands that need to be moved accurately and efficiently to specific points on 3D space and since the robot should adapt to any environment around it and avoid obstacles while attempting to carry or drop an object it was holding, the robot kinematics play a critical role in our project that can't be neglected by any means.

Section content:

In this section we will provide all the steps taken to establish the kinematics of our robot with detailed explanation of the problems faced and how we solved them also we will provide more information on how the final results can be improved to increase the accuracy by which the robot hands approach the object.

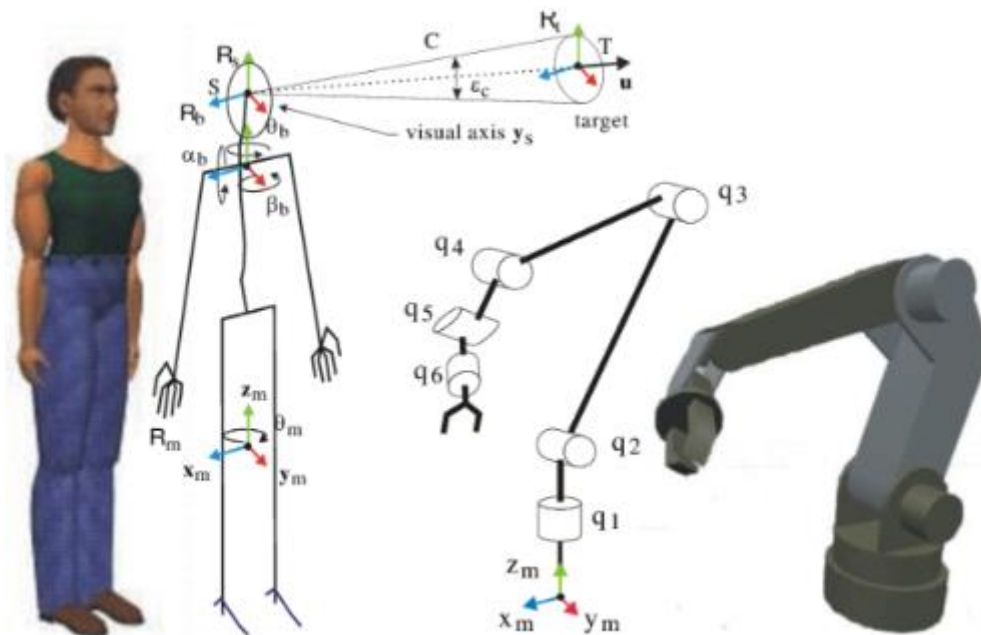


Figure 91

Robot URDF:

The URDF model for the robot is the first and one of the most important steps towards establishing the kinematics of the robot as it is the main file that describes the relation between the different links and joints of the robot which is then used by other software to calculate the kinematics of the robot.

What is URDF?

URDF, or Unified Robotics Description Format, is an XML specification used in academia and industry to model multibody systems such as robotic manipulator arms for manufacturing assembly lines and animatronic robots for amusement parks. URDF is especially popular with users of ROS, or Robotics Operating System—a framework that offers standard support for URDF models.

URDF generation steps:

1. The first step to generate the URDF was to identify what are the necessary parts that should be put in the URDF file and neglect the other unnecessary parts to reduce the design and computing overhead. We studied the design well before attempting to make an assembly in solid works. This helped reduce the number of parts considerably and solved one of the main problems faced while making the URDF which will be discussed in detail in the problems section.

By necessary parts we mean the parts that form the main dimensions of the robot which should be taken accurately or otherwise it will result in wrong calculation of the robot kinematics. On the other hand, the parts neglected were the parts responsible for giving the robot a nice shape like the covers and the other accessories.

2. We then started to assemble all needed parts using SolidWorks while putting in mind to join all the parts that move together as a single rigid part and keep only the moving joints that will later be used in the kinematics calculations.

After this step we had a very nice and optimized robot design with 5 joints and links on each hand.

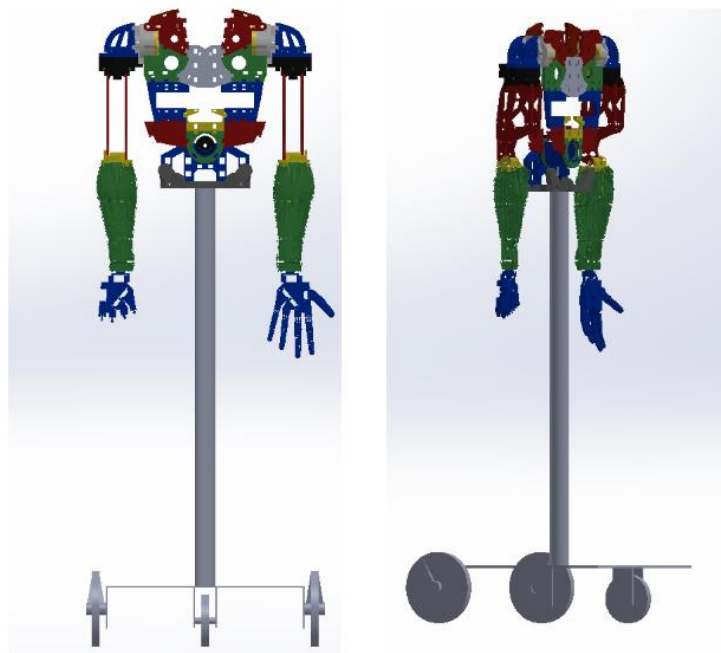


Figure 92

3. Next, we used the SW2URDF exporter to generate the URDF and all the other components necessary for working in ROS like the meshes launch files and config files.

Although this step was not hard as compared to the previous steps, but it took us many trials to get used to using the exporter as many things were required to be made before extracting the model and below are few things that were made:

- An axis of rotation should be added and selected manually for each revolute joint otherwise many problems may occur while exporting.
- It is better to select the global origin manually at the base of the robot as the position of everything else will be relative to this position and also it was important for the navigation to have a well-placed global origin.
- After many trials we found that giving all the frames generated automatically by the exporter the same orientation is better than leaving them as generated, this will make it easier to detect any errors in the URDF and change them manually if necessary.
- The joint limits were chosen to fit those of the actual robot for both the speed and position. This helped improve the real-world performance of the robot.

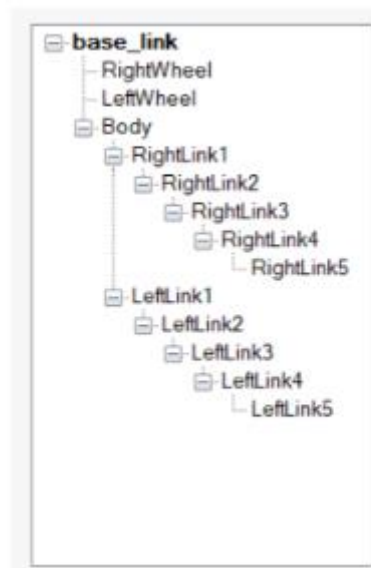


Figure 93

4. MoveIt

MoveIt is an open-source robotic manipulation platform that allows you to develop complex manipulation applications using ROS.

As the robot software mainly depends on ROS MoveIt was the best choice for establishing the robot kinematics because it can do various things and it is well documented which makes it easy to find solutions to the different errors that might occur.

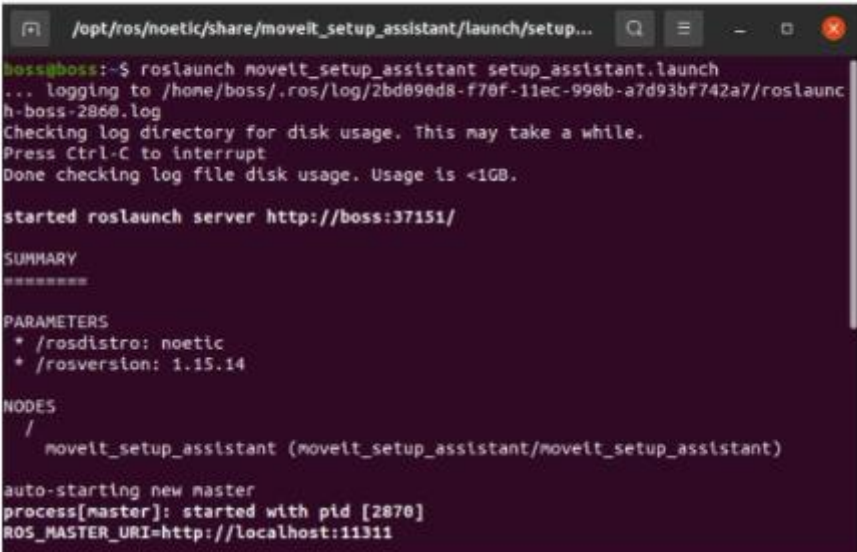
5. MoveIt setup assistant

The MoveIt Setup Assistant is a graphical user interface for configuring any robot for use with MoveIt. Its primary function is generating a Semantic Robot Description Format (SRDF) file for your robot. Additionally, it generates other necessary configuration files for use with the MoveIt pipeline.

Below is a step-by-step process to generate the necessary files using MoveIt setup assistant:

1. Launch MoveIt setup assistant using the following command from the terminal:

```
roslaunch moveit_setup_assistant setup_assistant.launch
```



```
boss@boss:~$ roslaunch moveit_setup_assistant setup_assistant.launch
... logging to /home/boss/.ros/log/2bd090d8-f70f-11ec-990b-a7d93bf742a7/roslaunch
h-boss-2800.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://boss:37151/

SUMMARY
=====

PARAMETERS
 * /rostdistro: noetic
 * /rosversion: 1.15.14

NODES
 /
   moveit_setup_assistant (moveit_setup_assistant/moveit_setup_assistant)

auto-starting new master
process[master]: started with pid [2870]
ROS_MASTER_URI=http://localhost:11311
```

Figure 94

2. Select the URDF file, which was previously generated, if everything is ok you should see the robot model to the right of the screen as the image below.



Figure 95

3. Click on the Self-Collisions pane selector on the left-hand side and click on the Generate Collision Matrix button. The Setup Assistant will work for a few seconds before presenting you the results of its computation in the main table.

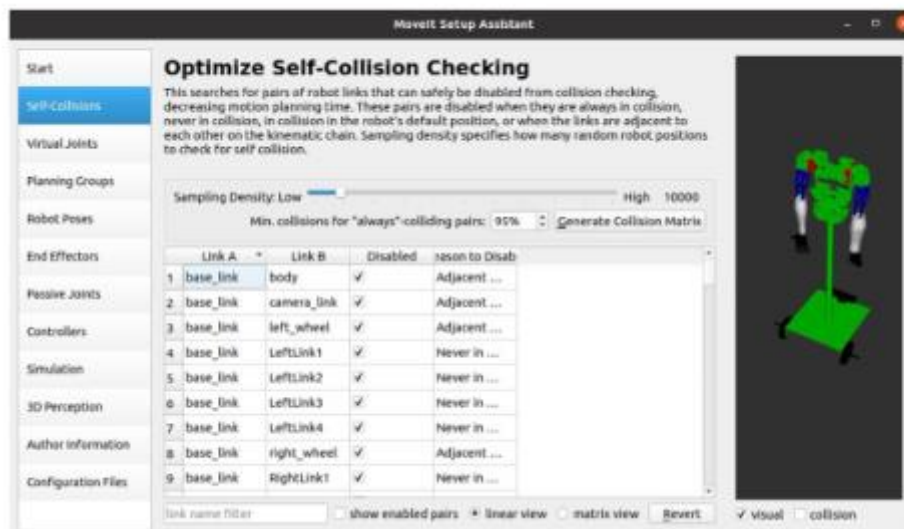


Figure 96

4. Add two planning groups, one for each hand and then add a third group that contains both hands.

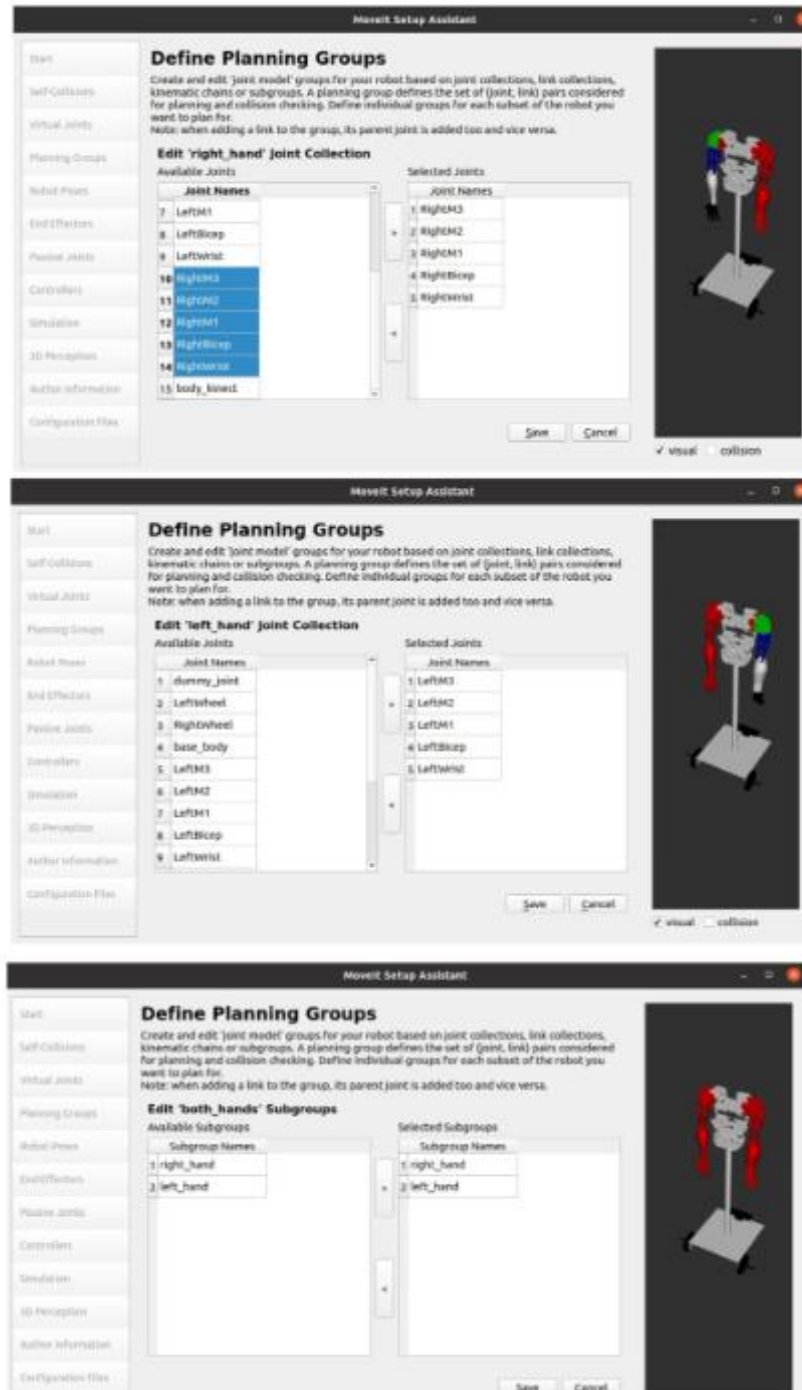


Figure 97



Figure 98

Important things to consider while generating the planning groups:

- The kinematics solver for both the right and left hands were generated specifically for them using IK_fast this will be discussed in detail at a later section.
- Both_hands group does not have a solver because it consists of the other two groups which have their own solvers.

5. Add fixed poses, if necessary, by moving the sliders for each joint until the required pose id reached

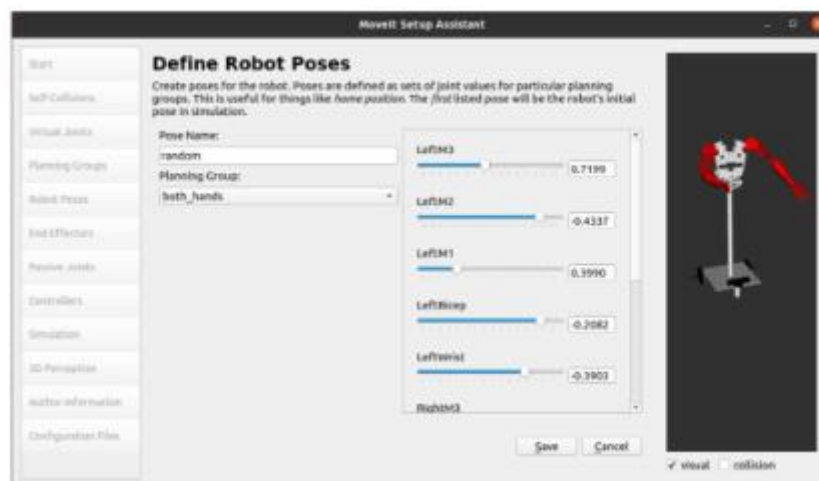


Figure 99

5. Add two end effectors one for each hand this step is very important for the inverse kinematics.



Figure 100

6. Add controllers for the two hands. As we are using servomotors, we used position joint controllers.



Figure 101

7. Finally generate the config file by pressing the generate package button. A good approach is to save the package in your catkin_ws this will make it easy to add other launch files for navigation and mapping in the same package to increase configurability.



Figure 102

To check if everything is ok run the following command in the terminal:

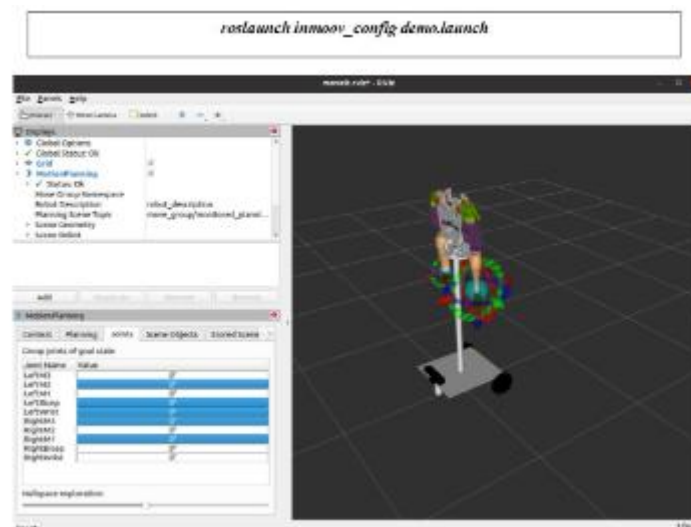


Figure 103

6. Hardware interface

In this section we will explain how to interface with the robot in real life along with the simulation. As mentioned in the hardware section each hand contains 5 position-controlled servo motors controlled via two Adafruit I2C to PWM converters along with an Arduino Mega.

By using Ros serial package, we send the required position for each motor to the Arduino which in turn makes sure that the motors reach their desired angles at the desired time.

7. Important parts of the Arduino code

```
/*call back function for subscriber*/
void GetAngles( const std_msgs::Float32MultiArrays angles)
{
    int i,j;
    for (i=9,j=2; i>=5; i--, j++)
    {
        RightSetAngles[i] = angles.data[j];
        Move(RIGHT, i, angles.data[j]);
    }

    for (i=9,j=7; i>=5; i--, j++)
    {
        LeftSetAngles[i] = angles.data[j];
        Move(LEFT, i, angles.data[j]);
    }
}
```

This function is called whenever the Arduino receives a new message on the "GetAngles" topic. It takes the angles indexed from 0 to 12 in the "angles" array and divide them into two arrays one for each hand according to the index of each motor in the two hands which is the same index in the I2C to PWM converter

Figure 104

```
/*create a ros subscriber to get the required velocity*/
ros::Subscriber<std_msgs::Float32MultiArray> AngleSub("/servo_angles", sGetAngles );
```

This line creates a ROS subscriber called "AngleSub" that subscribes to the topic "servo_angles" this topic contains the values that the motors should move to

```
void Move(int Side, int Part, int Angle)
{
    int pwm;
    if(Side == RIGHT)
    {
        CurrentRightAngles[Part] = Angle;
        pwm = Map(Angle, RightMinAngles[Part], RightMaxAngles[Part], RightMinPwms[Part], RightMaxPwms[Part]);
        Right.setPWM(Part, 0, pwm);
    }
    else if(Side == LEFT)
    {
        CurrentLeftAngles[Part] = Angle;
        pwm = Map(Angle, LeftMaxAngles[Part], LeftMinAngles[Part], LeftMaxPwms[Part], LeftMinPwms[Part]);
        Left.setPWM(Part, 0, pwm);
    }
}
```

This function is called inside the callback function it takes the values of the motors one by one and moves the motors to the required positions

```
int Map(float input, int in_max, int in_min, int out_max, int out_min)
{
    int result;
    result = ((input - in_min) * (out_max - out_min) / (in_max - in_min) + out_min);
    return result;
}
```

This function takes the float values of the angles in degrees and maps them to the corresponding PWM values which are previously set at the beginning of the code

Figure 105

When the robot moves in simulation it publishes the angles for each of the motors in a topic called "/joint_states" inside an array where each joint has its own index in the array. We subscribed to this topic and divided the array into two different separate arrays one for each hand, of course we had to convert the angles from radians to degrees before sending it to the Arduino to actually move the motors.

CHAPTER FIVE: COST ANALYSIS

Item name	Price of 1	Quantity	Total price
Xbox Kinect v1	700	2	1400
Arduino Mega	500	1	500
3D printing	-	-	8000
Filament for 3D print	450 / Kg	2 Kg	900
Motor Driver	800	1	800
Batteries	450	1	450
Sound sensors	60	1	60
Servo motor 10 kg.cm	160	12 (10 Already purchased)	320
IMU sensor	130	1	130
Extras	-	-	720
Total	-	-	13280

Table 6 Cost Analysis

CHAPTER SIX: CONCLUSION

1. We found that our robot can replace our role in hospitals, restaurants, hotels, factories in any repeated tasks that human workers do without intelligence or with limited intelligent thinking.
2. Using AI concepts and techniques can provide us opportunities to get the best advantage of our time and efforts to be used in managing, thinking, decision making instead of using it in repeated tasks that don't need high level intelligence.
3. Due to the human factor of our project (InMoov humanoid robot), we can depend on it on these tasks, but with more stable real time interaction and more rigid mechanical design of the legs, arms, and its dynamic motion.
4. With cameras, Kinect, speaker, mics we could resemble the human interaction as possible as can, but there is gap between these technologies and the real-world interactions, so we can say that the robots can replace our role in limited places not in all jobs and tasks.
5. despite the fact that there are still numerous minor subsystems that have not been completed, our team made every effort to work on them and came very close to finishing them. However, these subsystems will be left for the next teams to complete, as they have a high potential to be finished successfully.